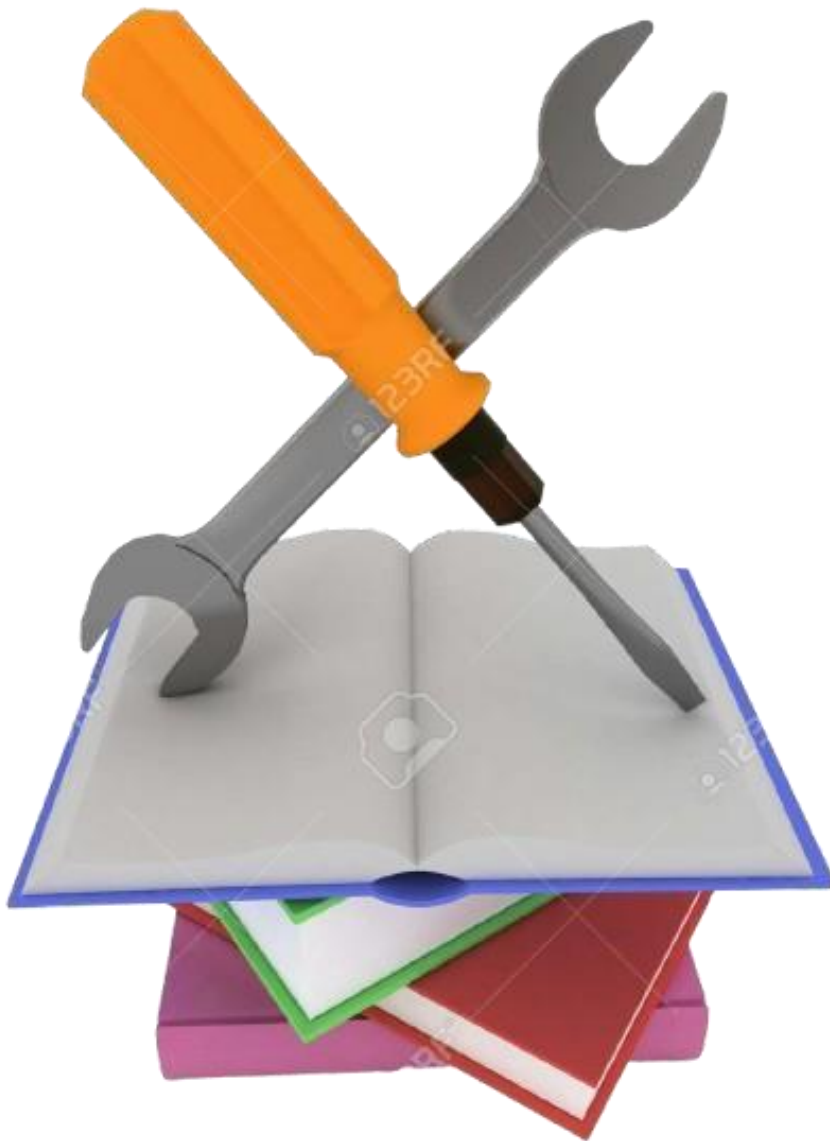


EQUIPO 8 MANUAL



TECNICO

Luis 317290967
Diego 319296738
Eduardo 319281998

Índice

Introducción	1
<i>Objetivo del proyecto</i>	1
<i>Visión general del proyecto</i>	1
<i>Cronograma de actividades</i>	2
<i>Información del proyecto</i>	8
Resumen tecnico	8
<i>Arquitectura</i>	8
<i>Estructura del codigo</i>	9
Condiciones necesarias del sistema	9
<i>Hardware</i>	9
<i>Software</i>	11
Modelos	11
<i>Modelos generados por blender o descargados de internet</i>	11
Animación	22
<i>Primera animación</i>	22
<i>Segunda animación</i>	23
<i>Tercera animación</i>	24
<i>Cuarta animación</i>	25
Código	
<i>Inclusiones y Variables Globales</i>	25
<i>Función para Renderizar un Modelo con Movimiento Vertical</i>	26
<i>Prototipos de Funciones para Interacción y Animación en la Escena</i>	26
<i>Inicialización de Variables para el Control de Cámara e Interacción</i>	26
<i>Variables para Control de Iluminación y Recorridos en la Escena</i>	27
<i>Control de Animaciones y Efectos de Iluminación en la Escena Gráfica</i>	27
<i>Variables y Keyframes para Animación de Remolino y Caída de Objetos en la Escena</i>	28
<i>Arreglo de Vértices y Normales de Cubo 3D</i>	30
<i>Inicialización y Configuración Básica de la Ventana y Shaders en OpenGL</i>	31
<i>Inicialización de Modelos 3D y Preparación de Keyframes para la Escena</i>	33
<i>Configuración de Buffers y Atributos de Vértices en OpenGL</i>	34
<i>Configuración de Unidades de Textura y Matriz de Proyección en OpenGL</i>	34
<i>Bucle Principal para Control de Animaciones y Eventos en la Escena</i>	34
<i>Configuración de Iluminación y Parámetros de Shading en la Escena 3D</i>	36
<i>Configuración Dinámica de Luces Puntuales y Spotlight en la Escena 3D</i>	37
<i>Lógica de Recorrido Automático de Cámara en la Escena 3D</i>	38

<i>Carga y Renderizado de Modelos 3D del Laboratorio Actual y Renovado</i>	39
<i>Renderizado y Animación de Sillas Viejas en la Escena 3D</i>	42
<i>Animación y Renderizado de Sillas Nuevas con Keyframes</i>	43
<i>Control de Movimiento de Cámara y Ajuste de Iluminación con Teclado</i>	44
<i>Gestión de Eventos de Teclado y Ratón para Control de Animaciones y Cámara</i>	45
Experiencia adquirida	47
Referencias	48

Introducción

Objetivo del proyecto

El proyecto tiene como objetivo principal desarrollar una experiencia interactiva a través de un recorrido virtual que permita visualizar detalladamente tanto la configuración actual como una propuesta integral de remodelación del Laboratorio de Microcomputadoras perteneciente a la Facultad de Ingeniería de la Universidad Nacional Autónoma de México (UNAM). Esta plataforma interactiva brindará a los usuarios la posibilidad de explorar el laboratorio desde múltiples perspectivas, comparar el estado actual con la nueva propuesta y apreciar claramente los cambios propuestos en aspectos como distribución del espacio, mobiliario, equipo tecnológico y adecuaciones estructurales necesarias. Con ello, se busca ofrecer una herramienta eficaz para la planeación, análisis y toma de decisiones sobre futuras mejoras en la infraestructura académica y tecnológica de este espacio universitario.

Visión general del proyecto

Para la realización del presente proyecto se emplearon diversos recursos tecnológicos y herramientas especializadas, con el objetivo de obtener una visualización clara y precisa tanto de las condiciones actuales del laboratorio como de la propuesta de renovación planteada.

Entre las herramientas utilizadas destaca Blender, un software de modelado tridimensional y animación gráfica. Este programa permitió construir de manera detallada los modelos en 3D de las instalaciones del laboratorio, incluyendo el mobiliario, equipos informáticos, elementos estructurales y distribución espacial. Además, Blender fue esencial para generar animaciones preliminares y visualizaciones realistas que ilustran tanto el estado actual del laboratorio como las modificaciones propuestas.

También se utilizó Visual Studio en combinación con el entorno gráfico OpenGL. Esta herramienta sirvió como plataforma principal para el desarrollo de la aplicación interactiva, mediante la cual los usuarios pueden explorar y navegar libremente por el recorrido virtual del laboratorio. OpenGL se empleó para implementar gráficos interactivos en tiempo real, gestionar las transformaciones visuales, efectos de iluminación, texturizado, animaciones, y permitir una experiencia fluida y realista del espacio virtual creado.

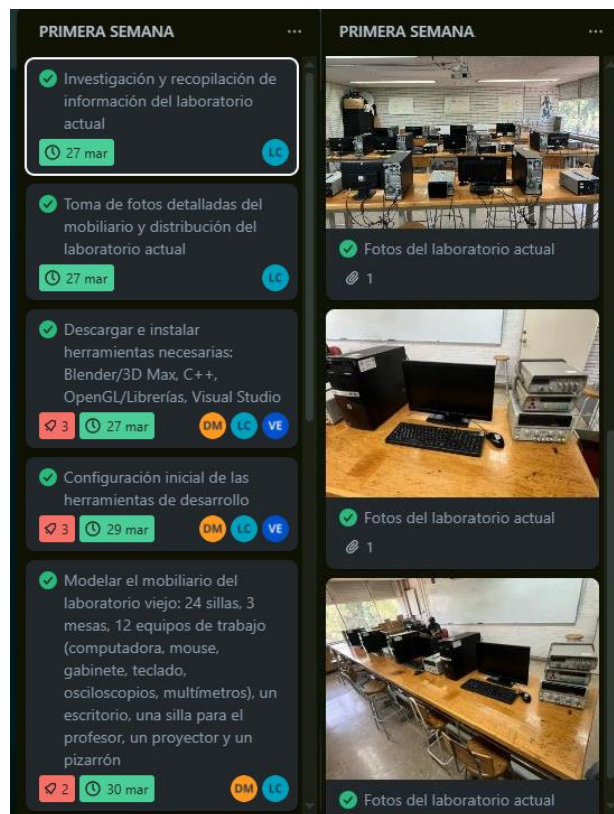
Finalmente, GitHub se incorporó al proyecto como sistema de control de versiones y colaboración. Esta plataforma permitió mantener un registro organizado y seguro de todas las etapas del desarrollo del software y los modelos gráficos utilizados, facilitando la colaboración eficiente entre los diferentes integrantes del equipo.

Cronograma de actividades

Para organizar y dar seguimiento a todas nuestras tareas del proyectos, elegimos la plataforma Trello como nuestra herramienta principal. Trello nos permite visualizar nuestro cronograma de actividades de manera clara y flexible, a través de tableros, listas y tarjetas que facilitan la colaboración en equipo

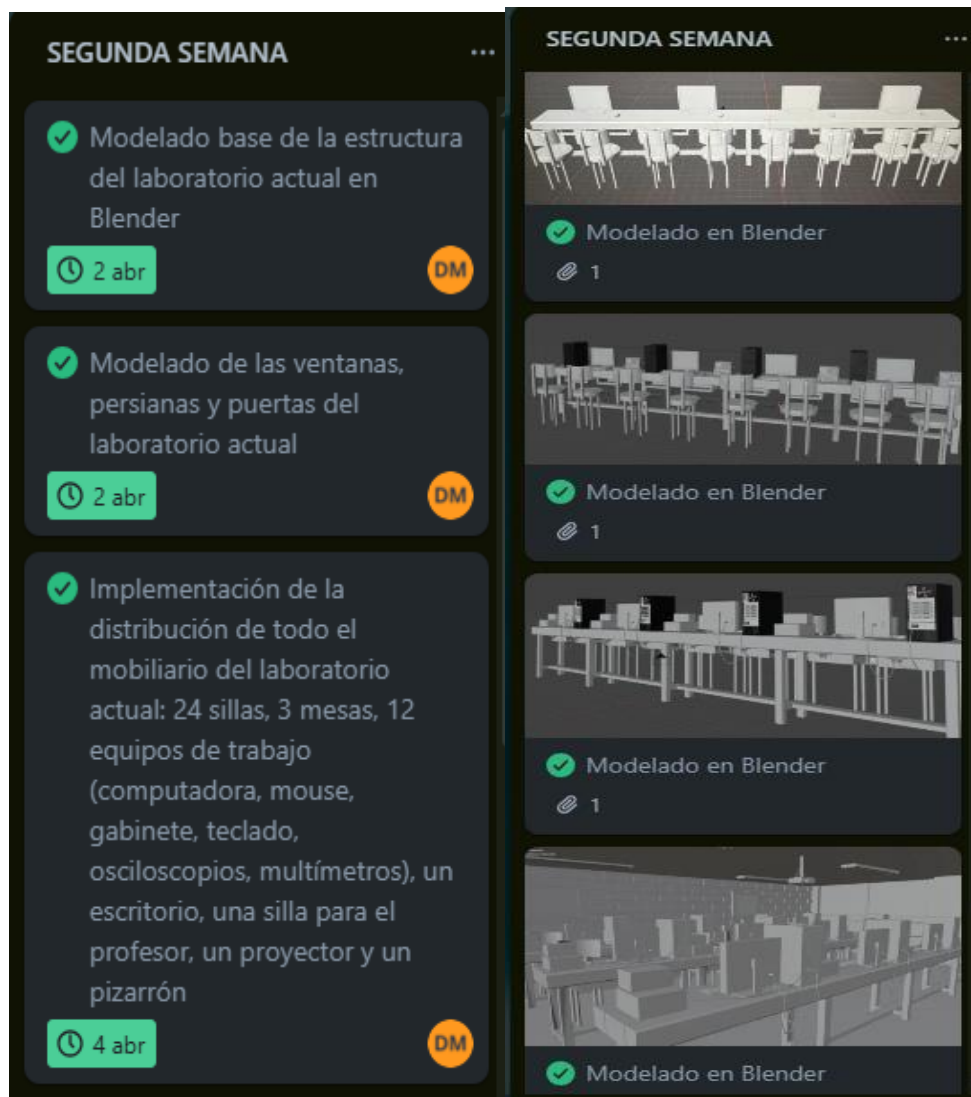
- **1er semana**

Durante la primera semana, el equipo se centró en recopilar y documentar toda la información necesaria sobre el laboratorio actual, incluyendo la toma de fotos detalladas del mobiliario y la distribución del espacio, así como en instalar y configurar las herramientas de desarrollo requeridas (como Blender, 3D Max, C++, OpenGL, Visual Studio) y comenzar el modelado digital del mobiliario existente. Todas estas actividades permitieron sentar las bases y preparar el entorno de trabajo necesario para el desarrollo del proyecto.



- **2da semana**

Durante la segunda semana, el equipo se enfocó en avanzar con el modelado digital del laboratorio actual utilizando Blender, realizando la base de la estructura, así como el modelado de ventanas, persianas y puertas. Además, implementaron la distribución completa del mobiliario del laboratorio, representando elementos como sillas, mesas, equipos de trabajo, escritorio del profesor, proyector y pizarrón, logrando así una representación virtual precisa y detallada del espacio físico existente.



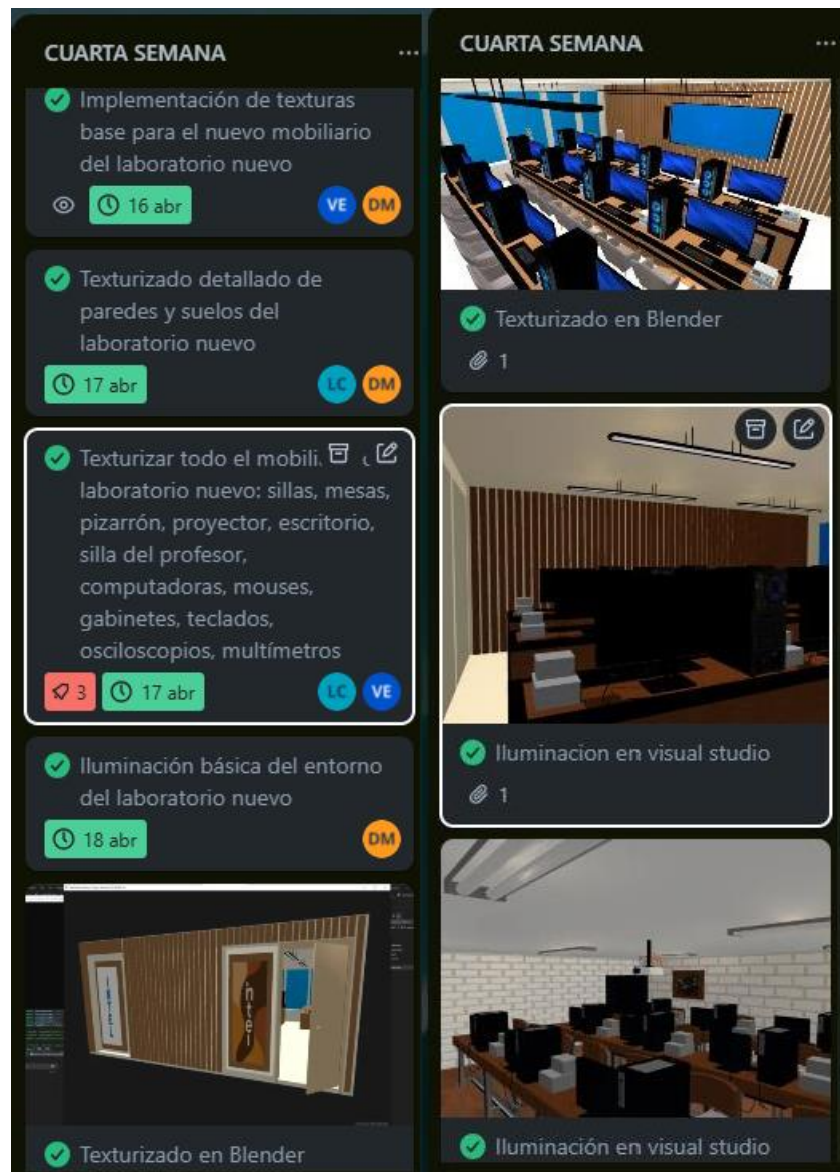
- **3er semana**

Durante la tercera semana, el equipo se dedicó al rediseño y mejora del laboratorio, creando en Blender una propuesta optimizada y moderna que incluyó el modelado del nuevo mobiliario y la mejora de los equipos de trabajo. Además, se realizó la texturización detallada tanto del mobiliario viejo como del nuevo, abarcando sillas, mesas, pizarrón, proyector, equipos de computo y demás elementos, logrando así una visualización más realista y completa de la propuesta del laboratorio actualizado.



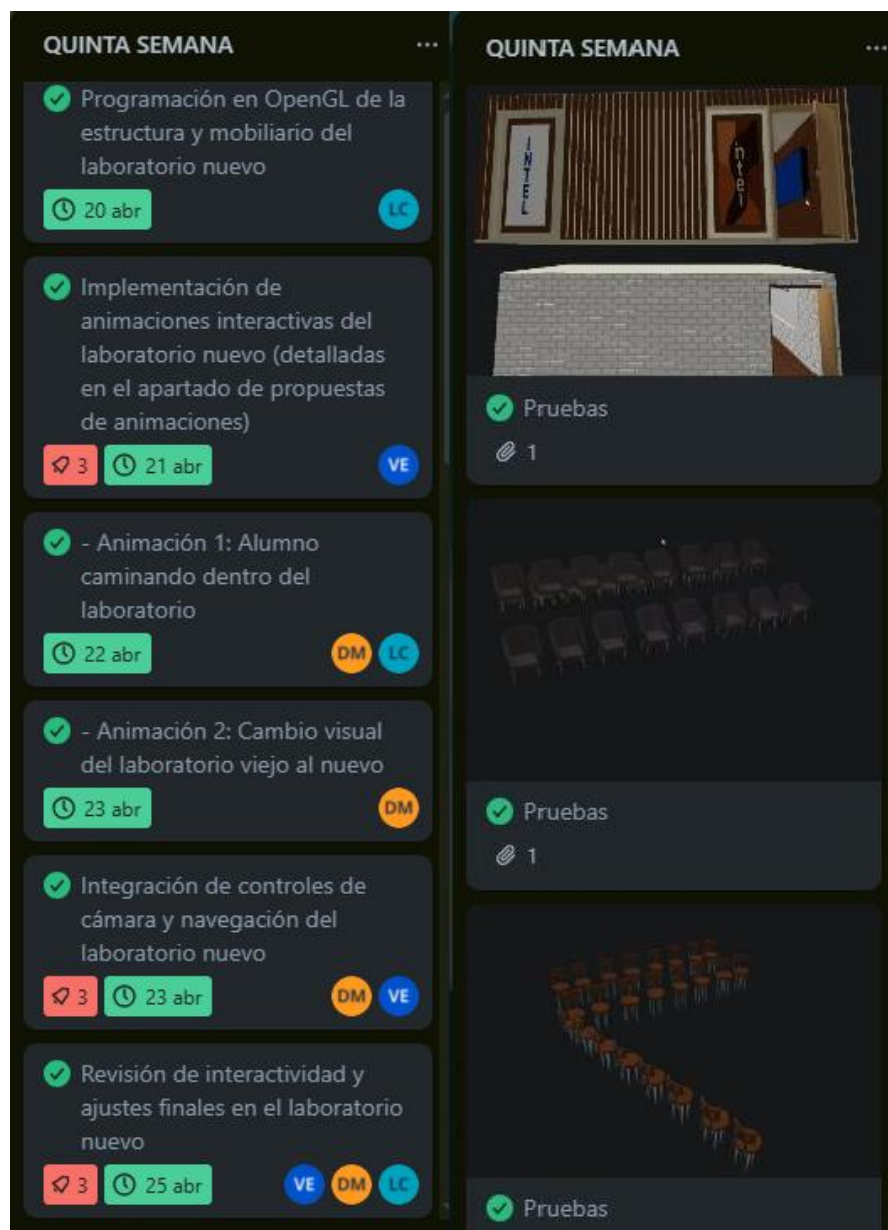
- **4ta semana**

Durante la cuarta semana, el equipo se enfocó en la implementación y detallado de texturas para el nuevo mobiliario, paredes y suelos del laboratorio, además de aplicar texturizado completo a todos los elementos del laboratorio nuevo, como sillas, mesas, pizarrón, proyector, equipos de cómputo y más. Finalmente, se trabajó en la iluminación básica del entorno virtual, logrando así una representación visual más realista y profesional del espacio propuesto en Blender y Visual Studio.



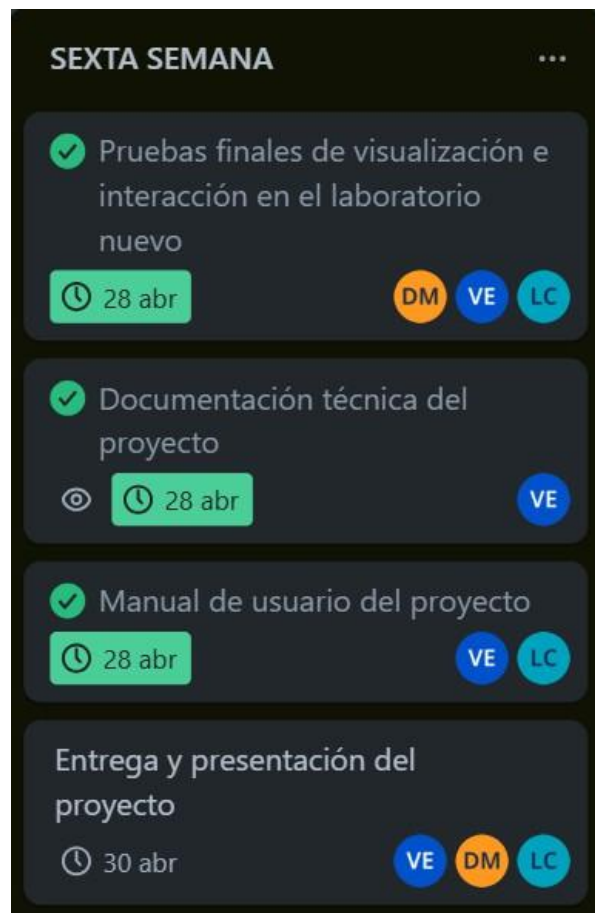
- **5ta semana**

Durante la quinta semana, el equipo se centró en el desarrollo e integración de animaciones interactivas dentro del laboratorio virtual, destacando la creación de una animación donde un alumno camina dentro del laboratorio y otra animación que permite visualizar el cambio entre el laboratorio viejo y el nuevo. Además, se trabajó en la integración de controles de cámara y navegación para mejorar la experiencia interactiva, y se realizaron pruebas y ajustes finales para asegurar que todas las animaciones funcionaran de manera fluida y realista



- **6ta semana**

Durante la sexta semana, el equipo se enfocó en realizar las pruebas finales de visualización e interacción en el laboratorio virtual, asegurándose de que todo funcionara correctamente. Además, se elaboró la documentación técnica y el manual de usuario del proyecto, dejando toda la información necesaria para el uso y mantenimiento del laboratorio. Finalmente, se preparó la entrega y presentación formal del proyecto, culminando así con la fase final del trabajo



La liga para poder observar el cronograma de mejor manera es la siguiente:

<https://trello.com/b/aNtqjwb6/proyecto-compu-grafica>

Información del proyecto

La información necesaria para la realización de los proyectos está en la siguiente liga:

https://github.com/DiegoRamirez25/PF_Laboratorio

Resumen técnico

Arquitectura

Para este proyecto se implementó una estructura unificada. Esta decisión se tomó considerando la naturaleza académica y experimental del trabajo, donde la prioridad era facilitar la integración y el manejo de las diferentes funcionalidades, especialmente durante el desarrollo y las pruebas.

La estructura unificada del proyecto implica que:

- Todas las funciones, variables globales, estructuras de animación y lógica de renderizado se encuentran concentradas en un solo archivo fuente.
- Los distintos módulos funcionales del sistema (manejo de modelos 3D, animación, gestión de cámara, iluminación, recorridos virtuales y control de eventos por teclado/mouse) se gestionan a través de funciones y estructuras definidas en el mismo archivo.
- El archivo principal interactúa directamente con las clases externas necesarias para la carga y renderizado de modelos (Model.h), manejo de shaders (Shader.h), cámara (Camera.h), y procesamiento de imágenes (stb_image.h, SOIL2.h), importando estos recursos desde sus respectivas bibliotecas.

Esta aproximación unificada permitió:

- Un rápido desarrollo y pruebas iterativas, ya que todas las partes del código están fácilmente accesibles y modificables en un mismo lugar.
- Facilitar la depuración y el seguimiento del flujo de ejecución, sobre todo al implementar animaciones complejas y la lógica de interacción.
- Integrar de manera directa el control de animaciones, la gestión de los recorridos virtuales y la actualización de los modelos dentro de un único ciclo de renderizado principal.

Estructura del código

El desarrollo de este proyecto se basa en una arquitectura integrada dentro de un solo archivo fuente, lo que facilita el control global y la gestión directa de todos los componentes del sistema. A continuación, se describe cómo está organizado el código:

1. **Inclusión de librerías y módulos:**
2. **Declaración de variables globales:**
3. **Definición de funciones principales:**
4. **Inicialización y carga de recursos:**
5. **Bucle principal de renderizado:**
6. **Gestión de eventos de entrada:**
7. **Finalización y limpieza:**

Condiciones necesarias del sistema

Para la correcta ejecución y visualización del proyecto de simulación y recorrido virtual en 3D del laboratorio, es necesario contar con una computadora que cumpla con ciertos requisitos mínimos de hardware y software. Estas especificaciones garantizan que el sistema pueda procesar y renderizar eficientemente los modelos tridimensionales, las animaciones y los efectos de iluminación implementados en el código.

Hardware

- **Procesador (CPU)**

El recurso mínimo es: Intel Core i5 de cuarta generación o superior / AMD Ryzen 3 o superior. El procesamiento de gráficos y la gestión de múltiples objetos 3D requieren un procesador capaz de manejar varias operaciones simultáneamente. Un CPU de estas características asegura que la aplicación pueda ejecutar las instrucciones necesarias sin provocar demoras o congelamientos, especialmente al manipular animaciones complejas y cálculos de física o transformaciones en tiempo real.

- **Memoria RAM**

El recurso mínimo es: 8 GB (16 GB recomendado). La RAM permite cargar tanto el programa principal como los modelos, texturas y librerías auxiliares en la memoria de trabajo. Una mayor cantidad de RAM es fundamental para evitar cuellos de botella durante la ejecución, sobre todo cuando se manejan múltiples objetos y recursos gráficos en escena. Si bien 8 GB es suficiente para ejecutar el programa, 16 GB proporcionan una mayor fluidez y capacidad de respuesta.

- **Sistema Operativo**

El recurso mínimo es: Windows 10/11 (64 bits). El código y las librerías empleadas (OpenGL, GLFW, GLEW, GLM, etc.) son compatibles con los principales sistemas operativos modernos. Se recomienda utilizar sistemas de 64 bits para aprovechar mejor la memoria disponible y la compatibilidad con las herramientas y dependencias del proyecto.

- **Tarjeta Gráfica (GPU)**

El recurso mínimo es: Compatible con OpenGL 3.3 o superior (NVIDIA GeForce GTX 750 / AMD Radeon R7 260X o equivalente, con al menos 2 GB de VRAM; 4 GB recomendados). La tarjeta gráfica es responsable del renderizado en tiempo real de los modelos 3D, las texturas, y las animaciones. Se requiere una GPU dedicada que soporte OpenGL 3.3 o versiones posteriores, ya que tecnologías previas pueden carecer de funciones esenciales para la iluminación avanzada, los materiales y las transformaciones implementadas en el proyecto. Más memoria de video (VRAM) mejora el manejo de texturas y modelos de mayor resolución.

- **Espacio en Disco Duro**

El recurso mínimo es: 2 GB libres. Este espacio es necesario para almacenar el código fuente, las librerías, los modelos 3D, las texturas y los recursos gráficos del proyecto. Se recomienda utilizar una unidad de estado sólido (SSD) para reducir los tiempos de carga de modelos pesados y acelerar el arranque del programa, aunque también puede funcionar en un disco duro tradicional (HDD).

- **Monitor y periféricos**

Monitor: Resolución mínima de 1920 x 1080. Una mayor resolución permite apreciar mejor los detalles del recorrido virtual y la interfaz gráfica del programa.
Periféricos: Teclado y ratón. Estos dispositivos son necesarios para interactuar con la aplicación, controlar la cámara virtual y activar las diferentes funciones y animaciones.

- **Acceso a Internet**

Se recomienda contar con acceso a Internet, al menos durante la instalación inicial, para poder descargar todas las dependencias y librerías necesarias para la compilación y ejecución del programa.

Software

Para desarrollar, compilar y ejecutar el proyecto de recorrido virtual y animación 3D del laboratorio, es necesario contar con el siguiente software:

- **Entorno de Desarrollo Integrado (IDE)**
Visual Studio (Windows) con configuración para desarrollo en OpenGL

Visual Studio proporciona un entorno de desarrollo robusto y profesional, adecuado para programar en C++. Además, permite integrar y gestionar fácilmente las librerías necesarias para trabajar con gráficos en 3D mediante OpenGL. La configuración adecuada de Visual Studio facilita la depuración, la gestión de dependencias y el manejo de proyectos grandes y complejos como este, garantizando así el óptimo desarrollo y ejecución del recorrido virtual.

Modelos

Modelos generados para el proyecto

Drive de los modelos:

<https://drive.google.com/drive/folders/1K3RUhuWEUohYiCcMjsCm2iosfkxKg8xd>

Modelo creado en blender

Sillas del laboratorio actual



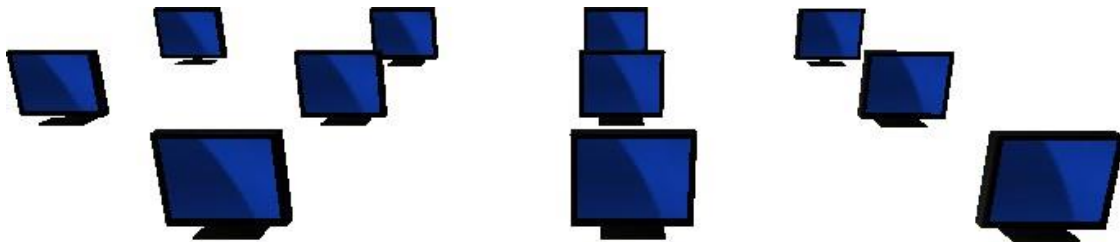
Modelo obtenido de la web. Liga: "Silla de club sencilla" (<https://skfb.ly/6WZRr>) by EnzoDesigns is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>)

Sillas del laboratorio remodelado



Modelo creado en blender

Monitores del laboratorio actual



Modelo obtenido de la web. Liga: "Near Future Computer Monitor" (<https://skfb.ly/6ZWfq>) by Pricey1600 is licensed under Creative Commons Attribution-NonCommercial (<http://creativecommons.org/licenses/by-nc/4.0/>).

Monitores del laboratorio remodelado



Modelo obtenido de la web. Liga: [Sketchfab](https://skfb.ly/oPvBW)

Gabinetes del laboratorio actual



Modelo obtenido de la web. Liga: "Gaming Setup (low-poly)" (<https://skfb.ly/oPvBW>) by LagzDesign is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>).

Gabinetes del laboratorio remodelado



Modelo creado en blender

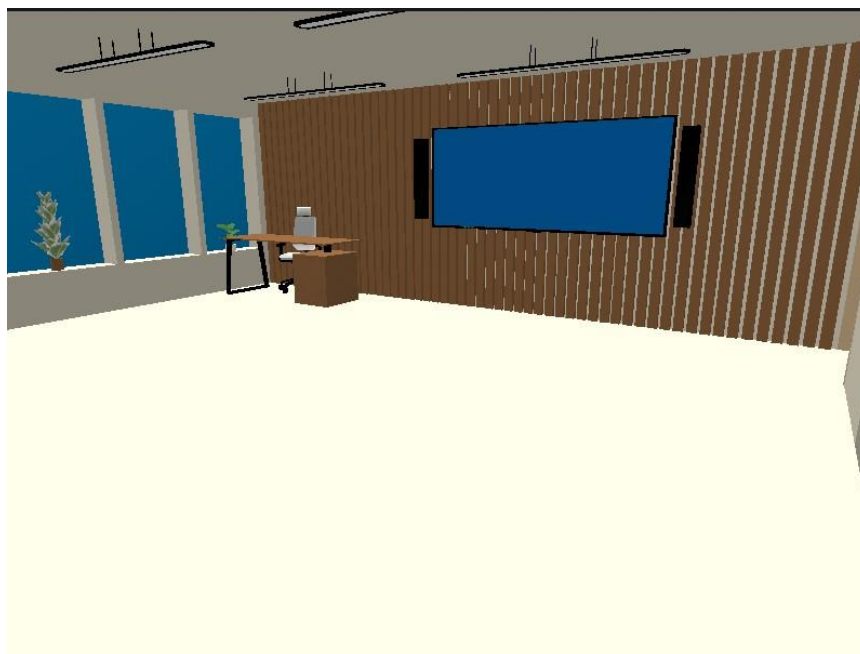
Cables de laboratorio actual y remodelado



Modelo creado en blender
Laboratorio actual



Modelo creado en blender
Laboratorio remodelado



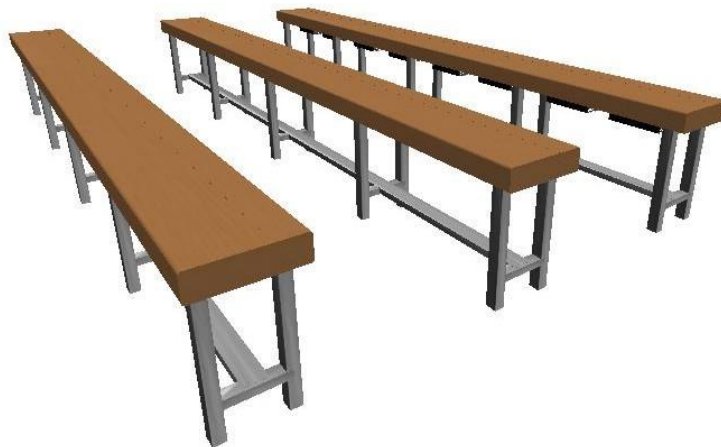
Modelo creado en blender

Osciloscopios y generador de funciones del laboratorio actual y el remodelado



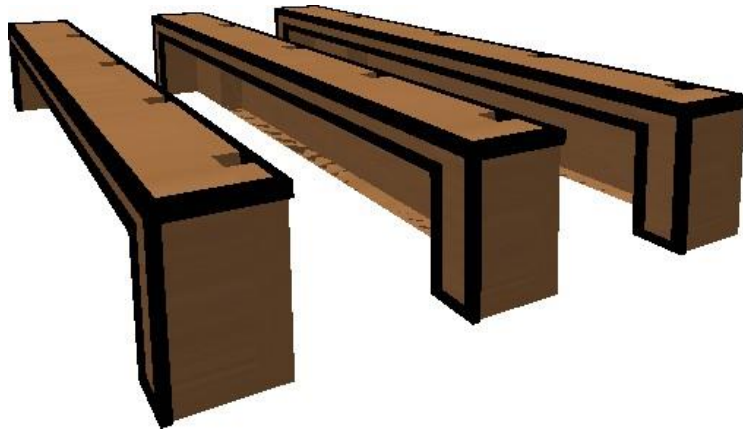
Modelo creado en blender

Mesas del laboratorio actual



Modelo creado en blender

Mesas del laboratorio nuevo



Modelo obtenido de la web. Liga: "Mouse" (<https://skfb.ly/DYRH>) by Microsoft is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>).

Mouses del laboratorio actual



Modelo creado en blender

Mouses del laboratorio



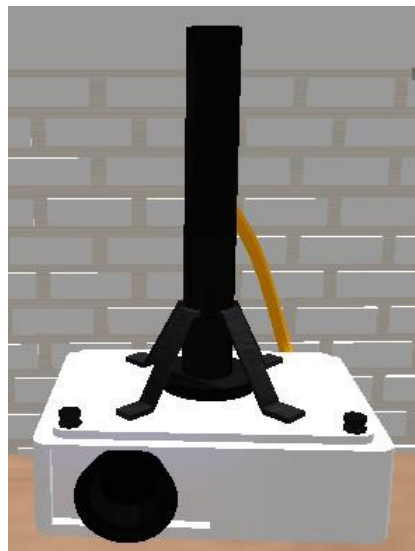
Modelo obtenido de la web. Liga: "Keyboard" (<https://skfb.ly/GXU9>) by thethieme is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>).

Teclados del laboratorio actual y del remodelado.



Modelo obtenido de la web. Liga: "Projector" (<https://skfb.ly/oqBor>) by bimadwiananto is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>).

Proyector del laboratorio actual.



Modelo obtenido de la web. Liga: "Gaming Setup (low-poly)" (<https://skfb.ly/oPvBW>) by LagzDesign is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>).

Mesa y silla del laboratorio actual.



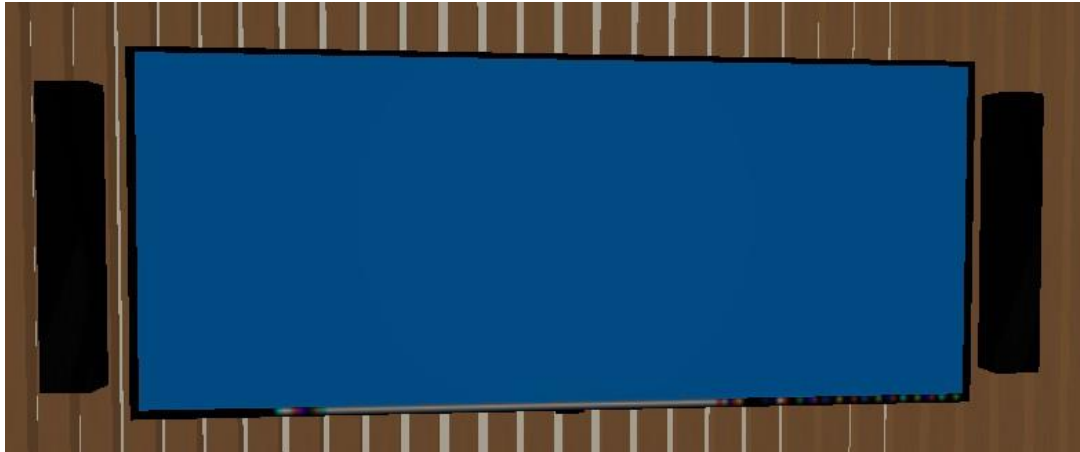
Modelo obtenido de la web. Liga: "Office chair" (<https://skfb.ly/onZ6s>) by AK STUDIO is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>).

Mesa y silla del laboratorio remodelado.



Modelo obtenido de la web. Liga: "TV" (<https://skfb.ly/oxAqG>) by CN Entertainment is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>).

Modelo de la televisión del laboratorio remodelado.



Modelo obtenido de la web. Liga: "Anime Classroom" (<https://skfb.ly/6xARy>) by ArielCastilleroV is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>).

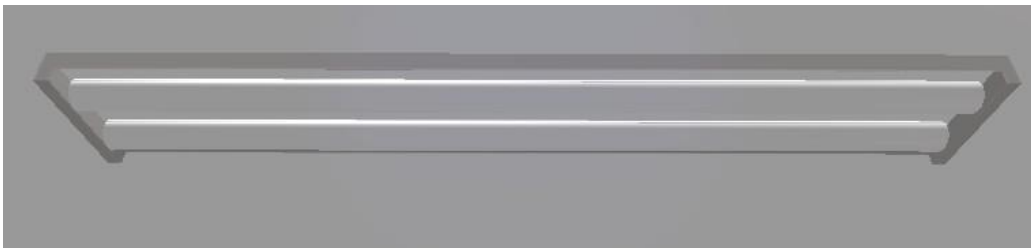
Lampara del laboratorio remodelado.



Modelo creado en blender
pizarrón del laboratorio actual



Modelo creado en blender
Lampara del laboratorio actual

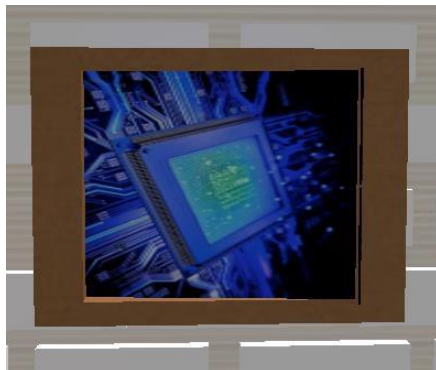


Modelo creado en blender
Cuadros del laboratorio actual



Modelo creado en blender

Cuadros del laboratorio actual



Modelo creado en blender

Planta de decoración del laboratorio remodelado



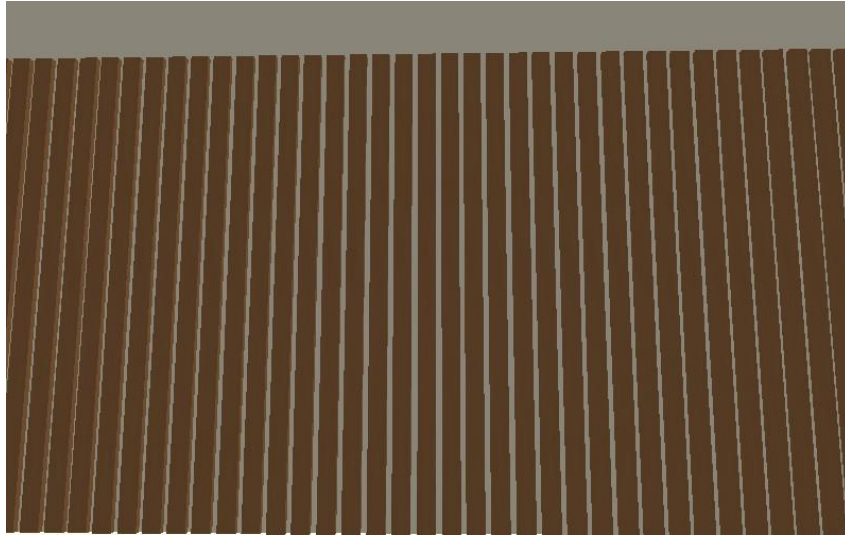
Modelo creado en blender

Planta de decoración del laboratorio remodelado



Modelo creado en blender

Madera de decoración del laboratorio remodelado



Animaciones

Primera animación.

La primera animación del video representa de forma creativa la transición entre el laboratorio actual y la propuesta de laboratorio remodelado. En esta secuencia, el nuevo laboratorio aparece descendiendo lentamente desde la parte superior de la escena. Este movimiento no es lineal ni rígido; en lugar de eso, se ha implementado un efecto de rebote al estilo "cartoon" o física estilizada, lo que le da una sensación dinámica y atractiva al descenso.

Finalmente, al tocar completamente la superficie, el nuevo laboratorio genera una especie de "aplastamiento" visual del laboratorio antiguo, simbolizando de manera gráfica y simbólica el reemplazo o la superposición del nuevo diseño sobre el anterior. Esta animación no solo introduce el cambio de entorno, sino que también capta la atención del espectador con una secuencia fluida y visualmente interesante.

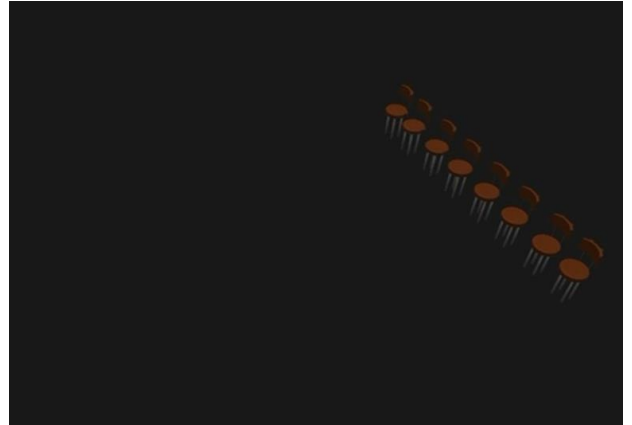
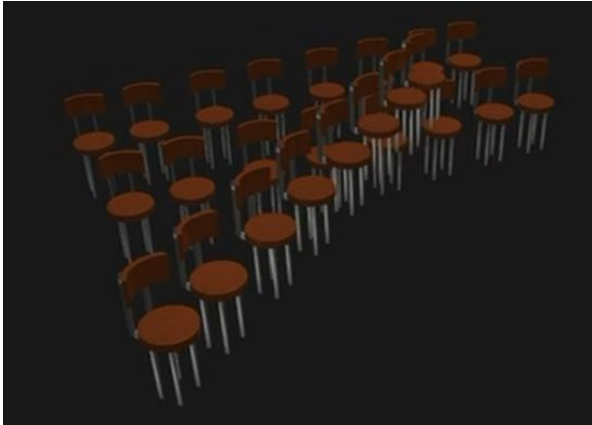


Segunda animación.

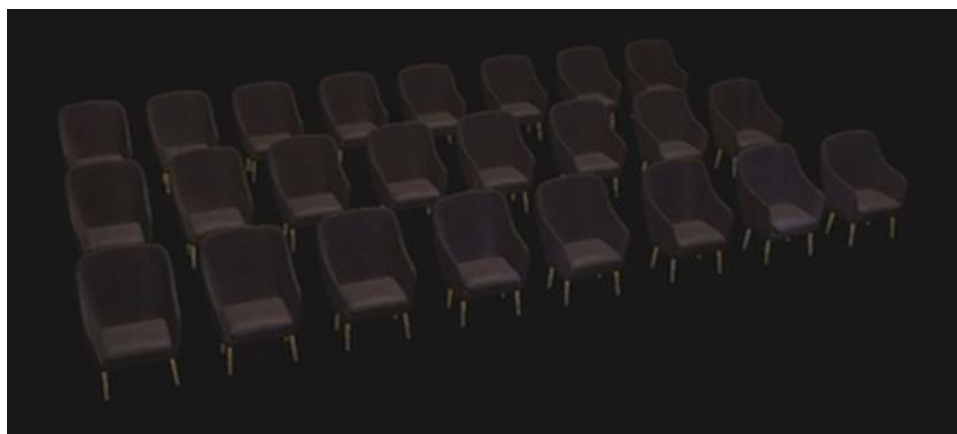
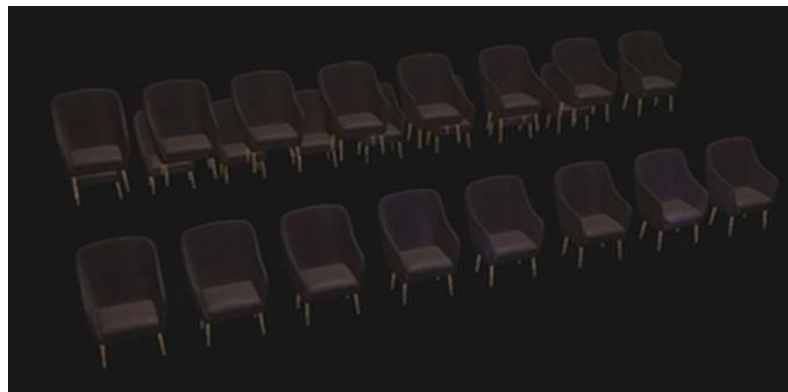
La segunda animación fue creada utilizando la técnica de *keyframe*. Estas animaciones representan la transición entre las sillas del laboratorio actual y las nuevas sillas del laboratorio remodelado.

En la primera parte de la secuencia, las sillas antiguas desaparecen progresivamente fila por fila, realizando varios giros de 360 grados que refuerzan visualmente la idea de cambio y movimiento. Posteriormente, las nuevas sillas hacen su entrada en escena con un efecto de rebote, creando la ilusión de que saltan ligeramente antes de acomodarse en sus posiciones definitivas dentro del espacio renovado.

Sillas del laboratorio actual



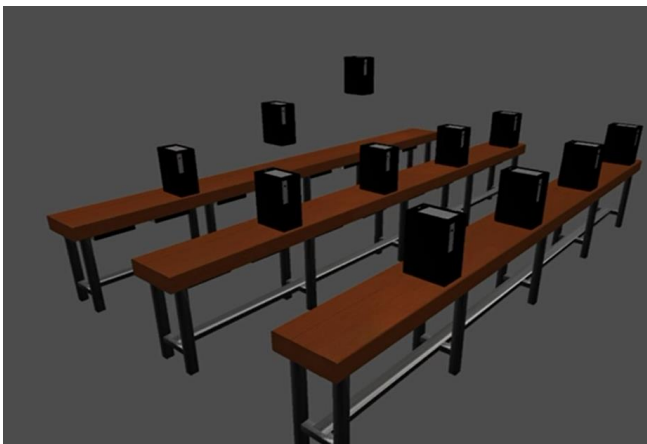
Sillas del laboratorio remodelado



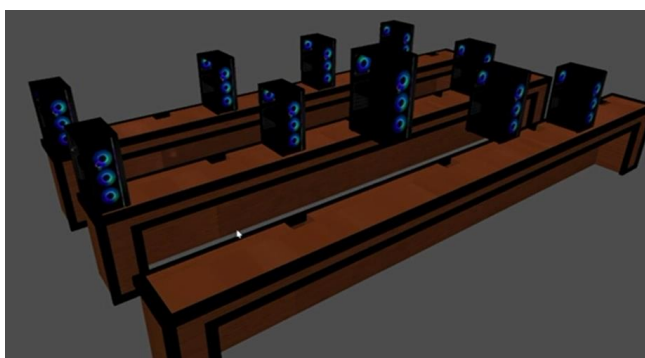
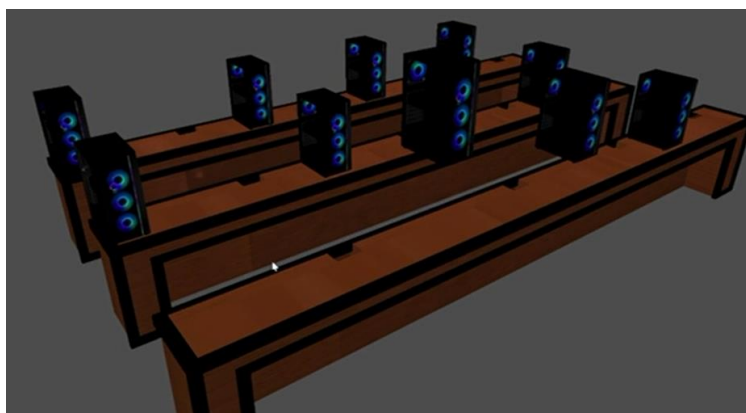
Tercera animación.

Las animaciones de los gabinetes se presentan de manera secuencial, apareciendo uno a uno con un efecto visual en el que parece que cada uno rebota al ingresar en la escena. Este efecto de rebote agrega dinamismo y fluidez a la animación, generando una sensación de movimiento en el espacio. Esta animación se implementará tanto para el laboratorio actual como para el remodelado, adaptándose a ambos entornos para mantener la coherencia visual y proporcionar una experiencia interactiva atractiva en ambos casos.

Animación de los gabinetes en laboratorio actual

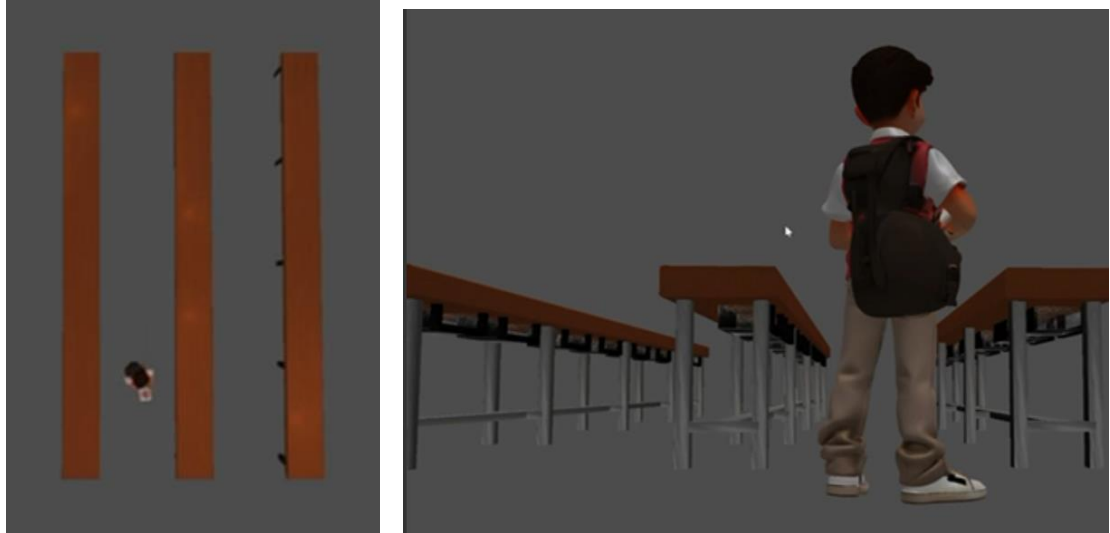


Animación de los gabinetes en laboratorio remodelado



Cuarta animación.

Esta animación simula el movimiento de un alumno caminando a través del salón de clases, siguiendo una trayectoria fluida y realista que refleja el paso natural de una persona. El proceso de caminata está cuidadosamente diseñado para mostrar la interacción del alumno con el entorno, como el movimiento de las piernas, el balance del cuerpo y la postura mientras avanza por el espacio. Además, la animación está acompañada del mobiliario.



Código

Para realizar el proyecto utilizamos el código base del previo 10, en esta sección se documentan todos los cambios relevantes que se hicieron en los archivos.

Inclusiones y Variables Globales

Bibliotecas Incluidas

- GLFW, GLEW: Manejan la ventana y las extensiones de OpenGL.
- GLM: Para operaciones matemáticas 3D (matrices y vectores).
- SOIL2, stb_image: Cargan texturas.
- Shader, Camera, Model: Son clases personalizadas para cargar shaders, controlar la cámara y manejar modelos 3D.

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include "stb_image.h"
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
#include "SOIL2/SOIL2.h"
#include "Shader.h"
#include "Camera.h"
#include "Model.h"
```

Función para Renderizar un Modelo con Movimiento Vertical

Define una función que se encarga de renderizar un modelo 3D en la escena aplicando una transformación de traslación vertical según una variable de posición, configurando la matriz de modelo para el shader, y luego dibujando el modelo con los parámetros actuales de animación y posición.

```
void renderBall(Shader& shader, Model& Ball) {  
  
    glm::mat4 model = glm::mat4(1.0f);  
  
    model = glm::translate(model, glm::vec3(0.0f, ballPosY, 0.0f));  
  
    glUniformMatrix4fv(glGetUniformLocation(shader.Program, "model"), 1, GL_FALSE,  
glm::value_ptr(model));  
  
    Ball.Draw(shader);  
}
```

Prototipos de Funciones para Interacción y Animación en la Escena

Declara los prototipos de varias funciones que gestionan la entrada del usuario por teclado y mouse, el movimiento dentro de la escena, la carga de datos de animación desde archivos y el dibujo de partes animadas de un modelo 3D, permitiendo así la interacción y animación en una aplicación gráfica.

```
void KeyCallback(GLFWwindow* window, int key, int scancode, int action, int mode);  
void MouseCallback(GLFWwindow* window, double xPos, double yPos);  
void DoMovement();  
void cargarKeyframesSillas(std::string archivoEntrada, std::string archivoSalida);  
void dibujarParteAnimada(Model& parte, float desfase, GLint modelLoc, Shader& shader);
```

Inicialización de Variables para el Control de Cámara e Interacción

Declara e inicializa las variables necesarias para gestionar la cámara y la entrada del usuario en una escena gráfica, incluyendo la posición inicial de la cámara, las coordenadas del último movimiento del mouse, un arreglo para registrar las teclas presionadas y una bandera que indica si es el primer movimiento de mouse, facilitando así el control interactivo de la visualización en la aplicación.

```
Camera camera(glm::vec3(3.0f, 10.0f, 0.0f));  
GLfloat lastX = WIDTH / 2.0;  
GLfloat lastY = HEIGHT / 2.0;  
bool keys[1024];  
bool firstMouse = true;
```

Variables para Control de Iluminación y Recorridos en la Escena

Declara e inicializa variables globales para gestionar la posición de una fuente de luz y banderas que controlan la activación de diferentes recorridos o animaciones dentro de la escena, permitiendo así manipular tanto la iluminación como la lógica de navegación o movimiento en la aplicación gráfica.

```
glm::vec3 lightPos(0.0f, 0.0f, 0.0f);  
bool active;  
bool recorridoActivo = false;  
bool recorridoActivo2 = false;
```

Control de Animaciones y Efectos de Iluminación en la Escena Gráfica

Declara e inicializa una serie de variables globales que se utilizan para controlar distintas animaciones y efectos dentro de una escena gráfica, incluyendo posiciones, velocidades, estados y factores que permiten simular la caída y rebote de un objeto desde una gran altura, afectado por la gravedad y con varios rebotes cada vez menores hasta que se detiene. Además, gestiona el efecto visual de aplastamiento y desvanecimiento de otro elemento al momento del impacto final, así como la manipulación de la transparencia y la iluminación mediante arreglos y vectores que representan posiciones de fuentes de luz y estados de los objetos animados, facilitando la manipulación dinámica tanto de la animación como de los efectos de iluminación en la aplicación.

```
glm::vec3 posLabNuevo = glm::vec3(-16.715f, 250.0f, 36.575f); // Más alto para caída larga  
float velocidadY = 0.0f;  
float gravedad = -1.0f;  
float reboteFactor = 0.8f;  
  
bool animacionActivaLabNuevo = false;  
bool animacionTerminada = false;  
  
int numRebotes = 0;  
int rebotesMaximos = 6;  
bool enRebote = true;  
bool impactoFinal = false;  
bool faseAplastamientoActiva = false;  
  
float escalaY_LabViejo = 1.0f;  
float alphaLabViejo = 1.0f; // transparencia del lab viejo  
  
glm::vec3 posicionLabViejo = glm::vec3(0.0f, 0.0f, 0.0f); // El lab viejo está en el origen  
  
float tiempoReaccionImpacto = 0.0f;
```

```

glm::vec3 pointLightPositions[] = {

glm::vec3(2.656f, 14.766f, 14.766f), //Luces calidas
glm::vec3(2.656f, 14.766f, -5.8f),
glm::vec3(2.656f, 14.766f, -10.766f),
glm::vec3(16.73f, 14.766f, 19.53f),
glm::vec3(16.73f, 14.766f,
glm::vec3(16.73f, 14.766f, -5.52f),
glm::vec3(16.73f, 14.766f, -10.073f),
glm::vec3(31.739f, 14.766f, 19.53f),
glm::vec3(31.739f, 14.766f, 14.977f),
glm::vec3(31.739f, 14.766f, -5.52f),
glm::vec3(31.739f, 14.766f, -10.073f),
glm::vec3(28.66f, 7.83f, 20.44f), //Luces azules
glm::vec3(18.56f, 7.83f, 10.68f),
glm::vec3(18.56f, 7.83f, 0.97f),
glm::vec3(7.75f, 7.83f, -8.92f),
};

std::vector<glm::vec3> lucesRelativasLabNuevo = {

glm::vec3(-1.21f, 77.78f, 27.009f),
glm::vec3(-1.21f, 77.78f, 16.97f),
glm::vec3(16.35f, 77.78f, 27.009f),
glm::vec3(16.35f, 77.78f, 16.97f),
glm::vec3(31.74f, 77.78f, 27.009f),
glm::vec3(31.74f, 77.78f, 16.97f),
glm::vec3(-1.21f, 77.78f, 2.71f),
glm::vec3(-1.21f, 77.78f, -7.31f),
glm::vec3(31.74f, 77.78f, 2.71f),
glm::vec3(31.74f, 77.78f, -7.31f),
glm::vec3(30.49f, 60.16f, 21.73f),
glm::vec3(30.49f, 60.16f, 7.15f),
glm::vec3(30.49f, 60.16f, -8.23f),
glm::vec3(20.49f, 60.16f, 21.73f),
glm::vec3(20.49f, 60.16f, 7.15f),
glm::vec3(20.49f, 60.16f, -8.23f),
glm::vec3(8.65f, 60.16f, 21.73f),
glm::vec3(8.65f, 60.16f, 7.15f),
glm::vec3(8.09f, 67.75f, 24.80f),
glm::vec3(8.09f, 67.75f, 0.46f),
glm::vec3(19.33f, 67.75f, 11.92f),
glm::vec3(30.49f, 67.75f, -11.01f),
glm::vec3(-11.19f, 71.08f, 2.88f)
};

```

Variables y Keyframes para Animación de Remolino y Caída de Objetos en la Escena

Declara e inicializa las variables y estructuras necesarias para controlar dos tipos de animaciones de objetos en la escena: la animación de “remolino” para un conjunto de sillas viejas (definiendo sus cuadros de animación, estados, velocidades, desfases y posiciones en cada frame), y la animación de “caída y rebote” para un conjunto de sillas nuevas (con cuadros de animación que representan distintas etapas de caída, rebotes y reposo). Además, define las funciones que preparan los keyframes (cuadros clave) para cada animación, permitiendo simular movimientos coordinados, efectos de elevación, remolino, rebote y descanso de los objetos en la escena 3D.

```

/ === Variables para animación tipo remolino de sillas viejas ===

// Sillas Viejas (Remolino)

int frameSillasViejas1 = 0, frameSillasViejas2 = 0, frameSillasViejas3 = 0;
bool animarSillasViejas1 = false, animarSillasViejas2 = false, animarSillasViejas3 = false; int delayRemolino1 = 0,
delayRemolino2 = 0, delayRemolino3 = 0;
int velocidadRemolino = 10;
int desfaseRemolino2 = 10; // Frames de espera antes de empezar la 2da int desfaseRemolino3 =
20; // Frames de espera antes de empezar la 3ra

// Sillas Nuevas (Caída)

int frameSillasNuevas1 = 0, frameSillasNuevas2 = 0, frameSillasNuevas3 = 0;
bool animarSillasNuevas1 = false, animarSillasNuevas2 = false, animarSillasNuevas3 = false; int delayCaida1 = 0,
delayCaida2 = 0, delayCaida3 = 0;
int velocidadCaida = 5;
int desfaseCaida2 = 10;
int desfaseCaida3 = 20;

struct FrameRemolino {
glm::vec3 posicion; float
rotacionY;
float escala;
};

std::vector<FrameRemolino> keyframesRemolino;
bool animarSillasViejas = false;
int frameSillasViejas = 0; int
delayRemolino = 0;

void prepararKeyframesRemolino() { keyframesRemolino.clear();

// 🚩 Elevación inicial
keyframesRemolino.push_back({ glm::vec3(0.0f, 0.0f, 0.0f), glm::radians(0.0f), 1.0f });
keyframesRemolino.push_back({ glm::vec3(0.0f, 4.5f, 0.0f), glm::radians(0.0f), 1.0f });

// 🏠 Remolino principal (ya existente)
keyframesRemolino.push_back({ glm::vec3(2.0f, 4.5f, 0.0f), glm::radians(0.0f), 1.0f });
keyframesRemolino.push_back({ glm::vec3(1.6f, 4.0f, 1.2f), glm::radians(36.0f), 0.95f });
keyframesRemolino.push_back({ glm::vec3(1.0f, 3.5f, 1.6f), glm::radians(72.0f), 0.9f });
keyframesRemolino.push_back({ glm::vec3(0.0f, 3.0f, 2.0f), glm::radians(108.0f), 0.85f });
keyframesRemolino.push_back({ glm::vec3(-1.0f, 2.5f, 1.6f), glm::radians(144.0f), 0.8f });
keyframesRemolino.push_back({ glm::vec3(-1.6f, 2.0f, 1.2f), glm::radians(180.0f), 0.75f });
keyframesRemolino.push_back({ glm::vec3(-2.0f, 1.5f, 0.0f), glm::radians(216.0f), 0.7f });
keyframesRemolino.push_back({ glm::vec3(-1.6f, 1.0f, -1.2f), glm::radians(252.0f), 0.6f });
keyframesRemolino.push_back({ glm::vec3(-1.0f, 0.5f, -1.6f), glm::radians(288.0f), 0.5f });
keyframesRemolino.push_back({ glm::vec3(0.0f, 0.0f, -2.0f), glm::radians(324.0f), 0.4f });

// 🏠 NUEVOS 10 KEYFRAMES (segunda vuelta más pequeña y más baja)
keyframesRemolino.push_back({ glm::vec3(0.8f, -0.2f, -0.8f), glm::radians(360.0f), 0.35f });
keyframesRemolino.push_back({ glm::vec3(0.6f, -0.5f, -0.6f), glm::radians(396.0f), 0.3f });
keyframesRemolino.push_back({ glm::vec3(0.4f, -0.8f, -0.4f), glm::radians(432.0f), 0.25f });
keyframesRemolino.push_back({ glm::vec3(0.2f, -1.1f, -0.2f), glm::radians(468.0f), 0.2f });
keyframesRemolino.push_back({ glm::vec3(0.0f, -1.4f, 0.0f), glm::radians(504.0f), 0.15f });
keyframesRemolino.push_back({ glm::vec3(-0.2f, -1.7f, 0.2f), glm::radians(540.0f), 0.1f });
keyframesRemolino.push_back({ glm::vec3(-0.4f, -2.0f, 0.4f), glm::radians(576.0f), 0.08f });
keyframesRemolino.push_back({ glm::vec3(-0.6f, -2.3f, 0.6f), glm::radians(612.0f), 0.05f });
keyframesRemolino.push_back({ glm::vec3(-0.8f, -2.6f, 0.8f), glm::radians(648.0f), 0.02f });
keyframesRemolino.push_back({ glm::vec3(0.0f, -3.0f, 0.0f), glm::radians(684.0f), 0.01f });
}

```

```

// === Variables para animación de caída y rebote de sillas nuevas ===
// Estructura para keyframes de caída struct FrameCaida
{
glm::vec3 posicion; float
escala;
};

std::vector<FrameCaida> keyframesCaidaSillasNuevas;
bool animarSillasNuevas = false;
int frameSillasNuevas = 0;
int delayCaida = 0;
bool labYaAterrizo = false; // se activa cuando el lab ya está abajo

void prepararKeyframesCaidaSillasNuevas() {
keyframesCaidaSillasNuevas.clear();

glm::vec3 base = labYaAterrizo ? glm::vec3(11.417f, -14.0f, 60.445f) : glm::vec3(-16.715f, 250.0f, 36.575f);

// 🎨 Caída inicial
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 40.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 35.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 30.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 25.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 20.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 10.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 5.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base, 1.0f }); // Impacto fuerte

// • Primer rebote alto
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 4.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 1.5f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base, 1.0f });

// • Segundo rebote
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 2.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 0.8f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base, 1.0f });

// • Tercer rebote pequeño
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 1.0f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 0.4f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base, 1.0f });

// • Cuarto rebote mínimo
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 0.5f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base + glm::vec3(0.0f, 0.2f, 0.0f), 1.0f });
keyframesCaidaSillasNuevas.push_back({ base, 1.0f });

// Final estable
keyframesCaidaSillasNuevas.push_back({ base, 1.0f }); // Frame 22 (reposo total)
}

```

Arreglo de Vértices y Normales de Cubo 3D

Define un arreglo de vértices que representa la geometría de un cubo 3D para ser utilizado en gráficos por computadora con OpenGL. Cada vértice está compuesto por seis valores: los primeros tres corresponden a la posición (x, y, z) del vértice en el espacio 3D y los siguientes tres corresponden a la normal de ese vértice (usada para iluminación y cálculos de luz en los shaders). De esta forma, el arreglo contiene toda

la información necesaria para renderizar un cubo con iluminación realista en una escena 3D.

```
float vertices[] = {
-0.5f, -0.5f, -0.5f,  0.0f,  0.0f, -1.0f,
 0.5f, -0.5f, -0.5f,  0.0f,  0.0f, -1.0f,
 0.5f,  0.5f, -0.5f,  0.0f,  0.0f, -1.0f,
 0.5f,  0.5f, -0.5f,  0.0f,  0.0f, -1.0f,
-0.5f,  0.5f, -0.5f,  0.0f,  0.0f, -1.0f,
-0.5f, -0.5f, -0.5f,  0.0f,  0.0f, -1.0f,

-0.5f, -0.5f,  0.5f,  0.0f,  0.0f,  1.0f,
 0.5f, -0.5f,  0.5f,  0.0f,  0.0f,  1.0f,
 0.5f,  0.5f,  0.5f,  0.0f,  0.0f,  1.0f,
 0.5f,  0.5f,  0.5f,  0.0f,  0.0f,  1.0f,
-0.5f,  0.5f,  0.5f,  0.0f,  0.0f,  1.0f,
-0.5f, -0.5f,  0.5f,  0.0f,  0.0f,  1.0f,

-0.5f,  0.5f,  0.5f, -1.0f,  0.0f,  0.0f,
-0.5f,  0.5f, -0.5f, -1.0f,  0.0f,  0.0f,
-0.5f, -0.5f, -0.5f, -1.0f,  0.0f,  0.0f,
-0.5f, -0.5f, -0.5f, -1.0f,  0.0f,  0.0f,
-0.5f, -0.5f,  0.5f, -1.0f,  0.0f,  0.0f,
-0.5f,  0.5f,  0.5f, -1.0f,  0.0f,  0.0f,

 0.5f,  0.5f,  0.5f,  1.0f,  0.0f,  0.0f,
 0.5f,  0.5f, -0.5f,  1.0f,  0.0f,  0.0f,
 0.5f, -0.5f, -0.5f,  1.0f,  0.0f,  0.0f,
 0.5f, -0.5f, -0.5f,  1.0f,  0.0f,  0.0f,
 0.5f, -0.5f,  0.5f,  1.0f,  0.0f,  0.0f,
 0.5f,  0.5f,  0.5f,  1.0f,  0.0f,  0.0f,

-0.5f, -0.5f, -0.5f,  0.0f, -1.0f,  0.0f,
 0.5f, -0.5f, -0.5f,  0.0f, -1.0f,  0.0f,
 0.5f, -0.5f,  0.5f,  0.0f, -1.0f,  0.0f,
 0.5f, -0.5f,  0.5f,  0.0f, -1.0f,  0.0f,
 0.5f, -0.5f,  0.5f,  0.0f, -1.0f,  0.0f,
-0.5f, -0.5f, -0.5f,  0.0f, -1.0f,  0.0f,

-0.5f,  0.5f, -0.5f,  0.0f,  1.0f,  0.0f,
 0.5f,  0.5f, -0.5f,  0.0f,  1.0f,  0.0f,
 0.5f,  0.5f,  0.5f,  0.0f,  1.0f,  0.0f,
 0.5f,  0.5f,  0.5f,  0.0f,  1.0f,  0.0f,
-0.5f,  0.5f,  0.5f,  0.0f,  1.0f,  0.0f,
-0.5f,  0.5f, -0.5f,  0.0f,  1.0f,  0.0f
};
```

Inicialización y Configuración Básica de la Ventana y Shaders en OpenGL

El código prepara todo el entorno necesario para iniciar una ventana de renderizado 3D con OpenGL, ya que declara variables para controlar el tiempo entre cuadros y una función para calcular el promedio de posiciones 3D, inicializa la ventana y el contexto gráfico con GLFW y GLEW, configura los callbacks de entrada, verifica posibles errores de inicialización, define el área de dibujo y prepara los shaders necesarios para la iluminación en la escena.

```

// Deltatime
GLfloat deltaTime = 0.0f;           // Time between current frame and last frame GLfloat
lastFrame = 0.0f;                   // Time of last frame

glm::vec3 promedioPosiciones(const glm::vec3* posiciones, int offset) { glm::vec3 suma(0.0f);
    for (int i = 0; i < 24; ++i) {
        suma += posiciones[i + offset];
    }
    return suma / 24.0f;
}

int main()
{
    // Inicialización de GLFW glfwInit();
    GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "Proyecto Final | Equipo 8",
    nullptr, nullptr);
    glfwMakeContextCurrent(window);

    glewExperimental = GL_TRUE; glewInit();
    // Init GLFW
    // Set all the required options for GLFW
    /*glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
    glfwWindowHint(GLFW_RESIZABLE, GL_FALSE);*/
    if (nullptr == window)
    {
        std::cout << "Failed to create GLFW window" << std::endl; glfwTerminate();

        return EXIT_FAILURE;
    }

    glfwMakeContextCurrent(window);

    glfwGetFramebufferSize(window, &SCREEN_WIDTH, &SCREEN_HEIGHT);

    // Set the required callback functions glfwSetKeyCallback(window, KeyCallback);
    glfwSetCursorPosCallback(window, MouseCallback);

    // GLFW Options
    //glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);

    // Set this to true so GLEW knows to use a modern approach to retrieving function pointers and extensions
    glewExperimental = GL_TRUE;
    // Initialize GLEW to setup the OpenGL Function pointers if (GLEW_OK !=
    glewInit())
    {
        std::cout << "Failed to initialize GLEW" << std::endl; return
        EXIT_FAILURE;
    }

    // Define the viewport dimensions
    glViewport(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT);

    Shader lightingShader("Shader/lighting.vs", "Shader/lighting.frag");
    Shader lampShader("Shader/lamp.vs", "Shader/lamp.frag");

```

Inicialización de Modelos 3D y Preparación de Keyframes para la Escena

Esta porción de código declara e inicializa objetos Model para cargar los diferentes modelos 3D que representan los elementos del laboratorio actual y el laboratorio nuevo en la escena, como sillas, mesas, luces y otros muebles o equipos. Los modelos están organizados por versiones (actual y nuevo), estos se incluyen para poder ser usados y renderizados según las necesidades de la simulación o animación. Al final, se llama a la función prepararKeyframesRemolino() para establecer los cuadros clave de la animación de remolino de las sillas, preparando así la estructura de datos necesaria para ejecutar esta animación en la escena 3D.

```
//==-- LAB ACTUAL ==--
```

```
//Model lab((char*)"Models/LAB.obj");
//Model medicion((char*)"Models/MEDICION.obj");
//Model medicion2((char*)"Models/MEDICION2.obj");
//Model medicion3((char*)"Models/MEDICION3.obj");
//Model medicion4((char*)"Models/MEDICION4.obj");
//Model mesas((char*)"Models/MESAS.obj"); Model
sillas((char*)"Models/SILLAS.obj"); Model
sillas2((char*)"Models/SILLAS2.obj"); Model
sillas3((char*)"Models/SILLAS3.obj");
//Model monitores((char*)"Models/MONITORES.obj");
//Model cables((char*)"Models/CABLES.obj");
//Model cables2((char*)"Models/CABLES2.obj");
//Model cables3((char*)"Models/CABLES3.obj");
//Model mouse((char*)"Models/MOUSES.obj");
//Model mouse2((char*)"Models/MOUSES2.obj");
//Model mouse3((char*)"Models/MOUSES3.obj");
//Model teclado((char*)"Models/TECLADOS.obj");
//Model teclado2((char*)"Models/TECLADOS2.obj");
//Model teclado3((char*)"Models/TECLADOS3.obj");
//Model gabinete((char*)"Models/GABINETES.obj");
//Model gabinete2((char*)"Models/GABINETES2.obj");
//Model gabinete3((char*)"Models/GABINETES3.obj"); Model
torso_ar((char*)"Models/TORSO_AR.obj");
Model pata1_ar((char*)"Models/PATA1_AR.obj"); Model
pata2_ar((char*)"Models/PATA2_AR.obj"); Model
pata3_ar((char*)"Models/PATA3_AR.obj"); Model
pata4_ar((char*)"Models/PATA4_AR.obj"); Model
pata5_ar((char*)"Models/PATA5_AR.obj"); Model
pata6_ar((char*)"Models/PATA6_AR.obj");
```

```
///// ==-- LAB NUEVO ==--+
```

```
//Model labN((char*)"Models/LABN.obj");
/*Model medicionLN((char*)"Models/MEDICION_LN.obj"); Model
medicionLN2((char*)"Models/MEDICION_LN2.obj"); Model
medicionLN3((char*)"Models/MEDICION_LN3.obj"); Model
medicionLN4((char*)"Models/MEDICION_LN4.obj"); Model
mesasLN((char*)"Models/MESAS_LN.obj");
Model gabinetesLN((char*)"Models/GABINETES_LN.obj"); Model
gabinetesLN2((char*)"Models/GABINETES_LN2.obj"); Model
gabinetesLN3((char*)"Models/GABINETES_LN3.obj"); Model
monitorLN((char*)"Models/MONITORES_LN.obj"); Model
monitorLN2((char*)"Models/MONITORES_LN2.obj"); Model
monitorLN3((char*)"Models/MONITORES_LN3.obj"); Model
monitorLN4((char*)"Models/MONITORES_LN4.obj"); Model
tecladoLN((char*)"Models/TECLADOS_LN.obj"); Model
tecladoLN2((char*)"Models/TECLADOS_LN2.obj"); Model
tecladoLN3((char*)"Models/TECLADOS_LN3.obj"); Model
mouseLN((char*)"Models/MOUSES_LN.obj");
Model mouseLN2((char*)"Models/MOUSES_LN2.obj");
```

```

Model mouseLN3((char*)"Models/MOUSES_LN3.obj");
Model cablesLN((char*)"Models/CABLES_LN.obj");
Model cablesLN2((char*)"Models/CABLES_LN2.obj");
Model cablesLN3((char*)"Models/CABLES_LN3.obj");*/
Model sillanLN((char*)"Models/SILLAS_LN.obj");
Model sillanLN2((char*)"Models/SILLAS_LN2.obj");
Model sillanLN3((char*)"Models/SILLAS_LN3.obj");
prepararKeyframesRemolino();

```

Configuración de Buffers y Atributos de Vértices en OpenGL

Configura los buffers y atributos necesarios en OpenGL para renderizar un objeto 3D: crea y vincula un Vertex Array Object (VAO) y un Vertex Buffer Object (VBO), transfiere los datos de los vértices a la GPU, y define cómo se interpretan los datos del arreglo de vértices, especificando la posición y la normal de cada vértice, de modo que el objeto pueda ser dibujado correctamente y se puedan aplicar efectos de iluminación en la escena.

```

// First, set the container's VAO (and VBO)
GLuint VBO, VAO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
// Position attribute
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(GLfloat), (GLvoid*)0);
glEnableVertexAttribArray(0);
// normal attribute
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 *
sizeof(float)));
glEnableVertexAttribArray(1);

```

Configuración de Unidades de Textura y Matriz de Proyección en OpenGL

Activa el shader de iluminación, asigna las unidades de textura a los mapas difuso y especular del material (usando las texturas en las posiciones 0 y 1), y crea la matriz de proyección en perspectiva que define cómo se visualizará la escena 3D en la ventana, ajustando el campo de visión según el zoom de la cámara y las proporciones de la pantalla.

```

// Set texture units
lightingShader.Use();
glUniform1i(glGetUniformLocation(lightingShader.Program, "Material.difuse"), 0);
glUniform1i(glGetUniformLocation(lightingShader.Program, "Material.specular"), 1);

glm::mat4 projection = glm::perspective(camera.GetZoom(), (GLfloat)SCREEN_WIDTH /
(GLfloat)SCREEN_HEIGHT, 0.1f, 100.0f);

```

Bucle Principal para Control de Animaciones y Eventos en la Escena

Implementa el ciclo principal del programa gráfico (game loop), que se ejecuta continuamente hasta que el usuario cierra la ventana. Dentro del ciclo, calcula el tiempo transcurrido entre cuadros para mantener las animaciones y movimientos

sincronizados con el tiempo real, actualiza matrices de proyección para efectos de luz, atiende eventos de usuario (teclado y mouse), y controla la lógica de animación de diferentes objetos, como la secuencia y los retardos de las animaciones de las sillas viejas y nuevas, permitiendo así una ejecución fluida y coordinada de las animaciones y la interacción en la escena 3D.

```
// Game loop
while (!glfwWindowShouldClose(window))
{
    // Calculate deltatime of current frame
    GLfloat currentFrame = glfwGetTime();
    deltaTime = currentFrame - lastFrame;
    lastFrame = currentFrame;

    glm::mat4 lightProjection, lightView;
    glm::mat4 lightSpaceMatrix;

    float near_plane = 1.0f, far_plane = 50.0f;
    lightProjection = glm::ortho(-20.0f, 20.0f, -20.0f, 20.0f, near_plane,
far_plane);
    lightView = glm::lookAt(glm::vec3(20.0f, 20.0f, 20.0f), glm::vec3(0.0f),
glm::vec3(0.0, 1.0, 0.0));
    lightSpaceMatrix = lightProjection * lightView;

    // Check if any events have been activated (key pressed, mouse moved etc.)
    and call corresponding response functions
    glfwPollEvents();
    DoMovement();

    if (animarSillasViejas1)
        delayRemolino1++;
        if (delayRemolino1 >= velocidadRemolino) {
            frameSillasViejas1++;
            delayRemolino1 = 0;
            // Activa la segunda cuando se cumpla el desfase
            if (!animarSillasViejas2 && frameSillasViejas1 >=
desfaseRemolino2) {
                animarSillasViejas2 = true;
            }
        }

    if (animarSillasViejas2) {
        delayRemolino2++;
        if (delayRemolino2 >= velocidadRemolino) { // Usa la misma
velocidad
            frameSillasViejas2++;
            delayRemolino2 = 0;
            if (!animarSillasViejas3 && frameSillasViejas2 >=
desfaseRemolino3) {
                animarSillasViejas3 = true;
            }
        }
    }

    if (animarSillasViejas3) {
        delayRemolino3++;
        if (delayRemolino3 >= velocidadRemolino) {
            frameSillasViejas3++;
            delayRemolino3 = 0;
        }
    }
}
```

```

        if (animarSillasNuevas1) {
            delayCaída1++;
            if (delayCaída1 >= velocidadCaída) {
                frameSillasNuevas1++;
                delayCaída1 = 0;
                if (frameSillasNuevas1 == desfaseCaída2)
animarSillasNuevas2 = true;
            }
        }
        if (animarSillasNuevas2) {
            delayCaída2++;
            if (delayCaída2 >= velocidadCaída) {
                frameSillasNuevas2++;
                delayCaída2 = 0;
                if (frameSillasNuevas2 == desfaseCaída3)
animarSillasNuevas3 = true;
            }
        }
        if (animarSillasNuevas3) {
            delayCaída3++;
            if (delayCaída3 >= velocidadCaída) {
                frameSillasNuevas3++;
                delayCaída3 = 0;
            }
        }
    }
}

```

Configuración de Iluminación y Parámetros de Shading en la Escena 3D

Activa el shader de iluminación y configura varios parámetros esenciales para el renderizado de la escena 3D: asigna la textura difusa, actualiza la posición de la cámara para la iluminación (útil para los efectos de reflejo especular), y construye una lista con todas las posiciones de luces en la escena (tanto del laboratorio viejo como del nuevo, aplicando los desplazamientos necesarios). Además, establece las propiedades de la luz direccional (dirección, componentes ambiente, difusa y especular) y calcula el color de una luz puntual de manera dinámica para lograr efectos animados en la iluminación. Todo esto prepara la escena para renderizarse con las condiciones de iluminación actualizadas en cada cuadro.

```

// Use cooresponding shader when setting uniforms/drawing objects
lightingShader.Use();

glUniform1i(glGetUniformLocation(lightingShader.Program, "diffuse"), 0);
glUniform1i(glGetUniformLocation(lightingShader.Program, "specular"), 1);

GLint viewPosLoc = glGetUniformLocation(lightingShader.Program, "viewPos");
glUniform3f(viewPosLoc, camera.GetPosition().x, camera.GetPosition().y,
camera.GetPosition().z);

std::vector<glm::vec3> lucesFinalesLabNuevo;
for (const auto& l : lucesRelativasLabNuevo) {
    lucesFinalesLabNuevo.push_back(l + posLabNuevo); // ¡Aquí aplicas
el desfase!
}

std::vector<glm::vec3> todasLasLuces;
// Agrega primero las del lab viejo
for (int i = 0; i < sizeof(pointLightPositions) / sizeof(glm::vec3); ++i) {

```

```

        todasLasLuces.push_back(pointLightPositions[i]);
    }

    // Luego las del lab nuevo (con desplazamiento aplicado)
    for (const auto& l : lucesFinalesLabNuevo) {
        todasLasLuces.push_back(l);
    }

    // Directional light
    glUniform3f(glGetUniformLocation(lightningShader.Program,
"dirLight.direction"), -0.2f, -1.0f, -0.3f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.ambient"),
0.6f, 0.6f, 0.6f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "dirLight.diffuse"),
0.6f, 0.6f, 0.6f);
    glUniform3f(glGetUniformLocation(lightningShader.Program,
"dirLight.specular"), 0.3f, 0.3f, 0.3f);

    // Point light 1
    glm::vec3 lightColor;
    lightColor.x = abs(sin(glfwGetTime() * Light1.x));
    lightColor.y = abs(sin(glfwGetTime() * Light1.y));
    lightColor.z = sin(glfwGetTime() * Light1.z);

```

Configuración Dinámica de Luces Puntuales y Spotlight en la Escena 3D

Recorre todas las luces de la escena y configura sus parámetros en el shader de iluminación: para cada luz, define su posición y ajusta sus propiedades ambiente, difusa y especular según el tipo (cálidas, blancas, azules animadas para monitores o tenues para el laboratorio nuevo), además de los factores de atenuación (constant, linear, quadratic) para simular cómo se dispersa la luz. También configura una luz tipo spotlight (simulando, por ejemplo, un proyector apuntando al pizarrón) estableciendo su posición, dirección, ángulos de corte y características lumínicas. Finalmente, define el valor de brillo (shininess) del material para calcular los reflejos especulares, permitiendo así una iluminación dinámica y variada en la escena 3D.

```

// ===== LUCES DE TECHO Y MONITORES CON ANIMACIÓN EN MONITORES =====

    for (int i = 0; i < todasLasLuces.size(); i++) {
        std::string idx = std::to_string(i);
        std::string base = "pointLights[" + idx + "].";

        glUniform3f(glGetUniformLocation(lightningShader.Program, (base +
"position").c_str()),
                    todasLasLuces[i].x, todasLasLuces[i].y,
todasLasLuces[i].z);

        if (i <= 10) {
            // Luces cálidas
            glUniform3f(glGetUniformLocation(lightningShader.Program,
(base + "ambient").c_str()), 0.05f, 0.04f, 0.03f);
            glUniform3f(glGetUniformLocation(lightningShader.Program,
(base + "diffuse").c_str()), 0.2f, 0.18f, 0.15f);
            glUniform3f(glGetUniformLocation(lightningShader.Program,
(base + "specular").c_str()), 0.3f, 0.25f, 0.2f);
        }
        else if (i >= 11 && i <= 32) {
            // Luces blancas

            glUniform3f(glGetUniformLocation(lightningShader.Program,
(base + "ambient").c_str()), 0.04f, 0.03f, 0.02f);

```

```

        glUniform3f(glGetUniformLocation(lightningShader.Program, (base +
"diffuse").c_str()), 0.2f, 0.17f, 0.1f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, (base +
"specular").c_str()), 0.3f, 0.25f, 0.2f);
    }
    else if (i >= 33 && i <= 40) {
        // Luces azules animadas (monitores) float t =
        glfwGetTime();
        float flicker = abs(sin(t * 2.0f + i));

        glUniform3f(glGetUniformLocation(lightningShader.Program, (base +
"ambient").c_str()), 0.01f, 0.01f, 0.05f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, (base +
"diffuse").c_str()), 0.4f * flicker, 0.4f * flicker, 1.0f * flicker);
        glUniform3f(glGetUniformLocation(lightningShader.Program, (base +
"specular").c_str()), 0.5f * flicker, 0.5f * flicker, 1.0f * flicker);
    }
    else {
        // ♦ Luces del lab nuevo: cálidas y tenues
        glUniform3f(glGetUniformLocation(lightningShader.Program,
(base + "ambient").c_str()), 0.02f, 0.015f, 0.01f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, (base +
"diffuse").c_str()), 0.1f, 0.08f, 0.05f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, (base +
"specular").c_str()), 0.2f, 0.15f, 0.1f);
    }

    glUniform1f(glGetUniformLocation(lightningShader.Program, (base +
"constant").c_str()), 1.0f);
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base +
"linear").c_str()), 0.14f);
    glUniform1f(glGetUniformLocation(lightningShader.Program, (base +
"quadratic").c_str()), 0.07f);

    // Spotlight - Luz azul del proyector al pizarrón glUniform3f(glGetUniformLocation(lightningShader.Program,
"spotLight.position"), 15.176f, 4.2527f, 12.95f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.direction"), 0.0f, -0.391f, -
0.9204f);
    glUniform1f(glGetUniformLocation(lightningShader.Program, "spotLight.cutOff"), glm::cos(glm::radians(9.0f)));
    glUniform1f(glGetUniformLocation(lightningShader.Program, "spotLight.outerCutOff"),
glm::cos(glm::radians(13.0f)));
    glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.ambient"), 0.01f, 0.01f,
0.05f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.diffuse"), 0.0f, 0.0f, 1.0f);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.specular"), 0.3f, 0.3f, 1.0f);
    glUniform1f(glGetUniformLocation(lightningShader.Program, "spotLight.constant"), 1.0f);
    glUniform1f(glGetUniformLocation(lightningShader.Program, "spotLight.linear"),
0.09f);
    glUniform1f(glGetUniformLocation(lightningShader.Program,
"spotLight.quadratic"), 0.032f);

    // Set material properties

    glUniform1f(glGetUniformLocation(lightningShader.Program, "material.shininess"), 5.0f);

```

Lógica de Recorrido Automático de Cámara en la Escena 3D

Implementa el recorrido automático de la cámara alrededor de dos escenarios: cuando recorridoActivo está habilitado, la cámara se mueve en círculo alrededor de un punto central en el laboratorio viejo, simulando una vista panorámica giratoria; y

cuando recorridoActivo2 está activo, la cámara da vueltas en un radio diferente y a mayor altura, enfocando el laboratorio nuevo. En ambos casos, la posición de la cámara y la dirección hacia la que mira se actualizan en cada cuadro usando funciones trigonométricas y el tiempo transcurrido, generando un efecto de “tour virtual” suave y continuo por la escena.

```
// ----- RECORRIDO LAB VIEJO -----

    if (recorridoActivo) {
        float t = glfwGetTime();
        float radius = 18.0f;
        float centerX = 20.0f;
        float centerZ = 6.35f;

        float camX = sin(t * 0.3f) * radius + centerX;
        float camZ = cos(t * 0.3f) * radius + centerZ;

        camera.SetPosition(glm::vec3(camX, 12.0f, camZ));
        camera.SetFront(glm::normalize(glm::vec3(centerX - camX, -0.5f,
centerZ - camZ)));
    }

    if (recorridoActivo2) {
        float t = glfwGetTime();
        float centerX = posLabNuevo.x + 16.0f;
        float centerZ = posLabNuevo.z + 3.0f;

        float radius = 24.5f;
        float camX = sin(t * 0.3f) * radius + centerX;
        float camZ = cos(t * 0.3f) * radius + centerZ;
        float camY = posLabNuevo.y + 70.0f; // Altura de cámara

        // Dirección: mirar hacia el centro pero con inclinación hacia
abajo en Y
        glm::vec3 direccion = glm::normalize(glm::vec3(centerX - camX, -
0.5f, centerZ - camZ));

        camera.SetPosition(glm::vec3(camX, camY, camZ));
        camera.SetFront(direccion);
    }
}
```

Carga y Renderizado de Modelos 3D del Laboratorio Actual y Renovado

Gestiona la carga y renderizado de los modelos 3D correspondientes tanto al laboratorio actual como al laboratorio nuevo en la escena gráfica. Para cada modelo (mesas, gabinetes, teclados, cables, monitores, mouse, etc.), crea una matriz de transformación (model), aplica una escala (en el caso del laboratorio actual) o una traslación a la posición del laboratorio nuevo, actualiza la matriz en el shader, y ajusta la propiedad de transparencia según sea necesario. Posteriormente, invoca el método Draw() de cada modelo para representarlos visualmente en la escena con iluminación. Esta sección permite alternar la visualización entre la disposición original y la renovada del laboratorio, reflejando los cambios espaciales y estructurales en la animación.

```
////// ===== LAB NUEVO =====
```

```

/*model = glm::mat4(1);
model = glm::translate(model, posLabNuevo); glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
glm::value_ptr(model));
glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"), 0); labN.Draw(lightningShader);*/

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//medicionLN.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//medicionLN2.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//medicionLN3.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//medicionLN4.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//mesasLN.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//gabinetesLN.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//gabinetesLN2.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//gabinetesLN3.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),

```

```

0);

    //monitorLN.Draw(lightingShader);

    //model = glm::mat4(1);
    //model = glm::translate(model, posLabNuevo);
    //glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "transparency"),

0);

    //monitorLN2.Draw(lightingShader);

    //model = glm::mat4(1);
    //model = glm::translate(model, posLabNuevo);
    //glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "transparency"),

0);

    //monitorLN3.Draw(lightingShader);

    //model = glm::mat4(1);
    //model = glm::translate(model, posLabNuevo);
    //glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "transparency"),

0);

    //monitorLN4.Draw(lightingShader);

    //model = glm::mat4(1);
    //model = glm::translate(model, posLabNuevo);
    //glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "transparency"),

0);

    //tecladoLN.Draw(lightingShader);

    //model = glm::mat4(1);
    //model = glm::translate(model, posLabNuevo);
    //glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "transparency"),

0);

    //tecladoLN2.Draw(lightingShader);

    //model = glm::mat4(1);
    //model = glm::translate(model, posLabNuevo);
    //glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "transparency"),

0);

    //tecladoLN3.Draw(lightingShader);

    //model = glm::mat4(1);
    //model = glm::translate(model, posLabNuevo);
    //glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "transparency"),

0);

    //mouseLN.Draw(lightingShader);

    //model = glm::mat4(1);
    //model = glm::translate(model, posLabNuevo);
    //glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "transparency"),

0);

    //mouseLN2.Draw(lightingShader);

    //model = glm::mat4(1);
    //model = glm::translate(model, posLabNuevo);
    //glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "transparency"),

0);

    //mouseLN3.Draw(lightingShader);

```

```

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//cablesLN.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//cablesLN2.Draw(lightningShader);

//model = glm::mat4(1);
//model = glm::translate(model, posLabNuevo);
//glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
//glUniform1i(glGetUniformLocation(lightningShader.Program, "transparency"),
0);

//cablesLN3.Draw(lightningShader);

```

Renderizado y Animación de Sillas Viejas en la Escena 3D

Controla la animación y el renderizado de tres conjuntos de sillas viejas en una escena 3D. Para cada conjunto de sillas, verifica si la animación correspondiente está activa y, de ser así, aplica transformaciones de posición, rotación y escala basadas en los keyframes de la animación tipo remolino, antes de dibujar el modelo de sillas usando el shader de iluminación. Si la animación no está activa, coloca cada conjunto de sillas en su posición base y las dibuja estáticamente. Esto permite visualizar las sillas en movimiento o en reposo dependiendo del estado de la animación.

```

// Sillas Viejas 1
// === SILLAS VIEJAS 1 ===
if (animarSillasViejas1 && frameSillasViejas1 < keyframesRemolino.size()) {
    FrameRemolino kf = keyframesRemolino[frameSillasViejas1];
    model = glm::mat4(1.0f);
    model = glm::translate(model, kf.posicion);
    model = glm::rotate(model, kf.rotacionY, glm::vec3(0, 1, 0));
    model = glm::scale(model, glm::vec3(kf.escala));
    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    sillas.Draw(lightningShader);
}
else if (!animarSillasViejas1) {
    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f)); //
Posición base

    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    sillas.Draw(lightningShader);
}

// === SILLAS VIEJAS 2 ===
if (animarSillasViejas2 && frameSillasViejas2 < keyframesRemolino.size()) {
    FrameRemolino kf = keyframesRemolino[frameSillasViejas2];

    model = glm::mat4(1.0f);
    model = glm::translate(model, kf.posicion);
    model = glm::rotate(model, kf.rotacionY, glm::vec3(0, 1, 0));
    model = glm::scale(model, glm::vec3(kf.escala));
    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    sillas2.Draw(lightningShader);
}
else if (!animarSillasViejas2) {
    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f)); //
Posición base

```

```

        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        sillas2.Draw(lightningShader);
    }

    // === SILLAS VIEJAS 3 ===
    if (animarSillasViejas3 && frameSillasViejas3 < keyframesRemolino.size()) {
        FrameRemolino kf = keyframesRemolino[frameSillasViejas3];
        model = glm::mat4(1.0f);
        model = glm::translate(model, kf.posicion);
        model = glm::rotate(model, kf.rotacionY, glm::vec3(0, 1, 0));
        model = glm::scale(model, glm::vec3(kf.escala));
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        sillas3.Draw(lightningShader);
    }
    else if (!animarSillasViejas3) {
        model = glm::mat4(1.0f);
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f)); //
Posición base

        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        sillas3.Draw(lightningShader);
    }
}

```

Animación y Renderizado de Sillas Nuevas con Keyframes

Se encarga de renderizar y animar tres modelos de sillas nuevas en la escena 3D. Utiliza listas de keyframes (`keyframesCaidaSillasNuevas`) para determinar la posición y escala de cada silla en cada cuadro de la animación, simulando su movimiento de caída. Si la animación de una silla está activa y no ha terminado, toma el frame correspondiente de la lista de keyframes y aplica sus transformaciones antes de dibujar la silla. Si la animación ha terminado o no está activa, la silla se dibuja en la posición y escala final definida por el último keyframe, asegurando así una transición suave entre animación y reposo en la escena.

```

// === SILLAS NUEVAS 1 ===

    if (animarSillasNuevas1 && frameSillasNuevas1 <
keyframesCaidaSillasNuevas.size()) {
        FrameCaida kf = keyframesCaidaSillasNuevas[frameSillasNuevas1];
        model = glm::mat4(1.0f);
        model = glm::translate(model, kf.posicion);
        model = glm::scale(model, glm::vec3(kf.escala));
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        sillasLN.Draw(lightningShader);
    }
    else if (!keyframesCaidaSillasNuevas.empty()) {
        FrameCaida kfFinal = keyframesCaidaSillasNuevas.back();
        model = glm::mat4(1.0f);
        model = glm::translate(model, kfFinal.posicion);
        model = glm::scale(model, glm::vec3(kfFinal.escala));
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        sillasLN.Draw(lightningShader);
    }

    // === SILLAS NUEVAS 2 ===

    if (animarSillasNuevas2 && frameSillasNuevas2 <
keyframesCaidaSillasNuevas.size()) {
        FrameCaida kf = keyframesCaidaSillasNuevas[frameSillasNuevas2];
        model = glm::mat4(1.0f);
        model = glm::translate(model, kf.posicion);
        model = glm::scale(model, glm::vec3(kf.escala));
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    }
}

```

```

        sillasLN2.Draw(lightningShader);
    }
    else if (!keyframesCaidaSillasNuevas.empty()) {
        FrameCaida kfFinal = keyframesCaidaSillasNuevas.back();
        model = glm::mat4(1.0f);
        model = glm::translate(model, kfFinal.posicion);
        model = glm::scale(model, glm::vec3(kfFinal.escala));
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        sillasLN2.Draw(lightningShader);
    }

    // === SILLAS NUEVAS 3 ===

    if (animarSillasNuevas3 && frameSillasNuevas3 <
keyframesCaidaSillasNuevas.size()) {
        FrameCaida kf = keyframesCaidaSillasNuevas[frameSillasNuevas3];
        model = glm::mat4(1.0f);
        model = glm::translate(model, kf.posicion);
        model = glm::scale(model, glm::vec3(kf.escala));
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        sillasLN3.Draw(lightningShader);
    }
    else if (!keyframesCaidaSillasNuevas.empty()) {
        FrameCaida kfFinal = keyframesCaidaSillasNuevas.back();
        model = glm::mat4(1.0f);
        model = glm::translate(model, kfFinal.posicion);
        model = glm::scale(model, glm::vec3(kfFinal.escala));
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        sillasLN3.Draw(lightningShader);
    }
}

```

Control de Movimiento de Cámara y Ajuste de Iluminación con Teclado

Se define la función `DoMovement`, la cual se encarga de manejar el movimiento de la cámara en la escena 3D y el ajuste de la posición de una de las luces puntuales (point lights) en respuesta a las teclas presionadas por el usuario. Si la variable `recorridoActivo` está activada, la función termina de inmediato (bloqueando el control manual). Si no, permite mover la cámara hacia adelante, atrás, izquierda o derecha usando las teclas W, S, A, D o las flechas del teclado, y también ajustar la posición en X, Y y Z de la primera luz puntual con las teclas T, G, Y, H, U y J. Esto facilita la exploración y personalización en tiempo real del entorno y la iluminación dentro de la escena gráfica.

```

void DoMovement()
{
    if (recorridoActivo) return;

    // Camera controls
    if (keys[GLFW_KEY_W] || keys[GLFW_KEY_UP])
    {
        camera.ProcessKeyboard(FORWARD, deltaTime);
    }
    if (keys[GLFW_KEY_S] || keys[GLFW_KEY_DOWN])
    {
        camera.ProcessKeyboard(BACKWARD, deltaTime);
    }

    if (keys[GLFW_KEY_A] || keys[GLFW_KEY_LEFT])
    {

```

```

        camera.ProcessKeyboard(LEFT, deltaTime);
    }

    if (keys[GLFW_KEY_D] || keys[GLFW_KEY_RIGHT])
    {
        camera.ProcessKeyboard(RIGHT, deltaTime);
    }
}

```

Gestión de Eventos de Teclado y Ratón para Control de Animaciones y Cámara

Implementa las funciones de callback para gestionar eventos de teclado y ratón en una aplicación de OpenGL con GLFW. La función `KeyCallback` detecta y responde a distintas teclas para cerrar la ventana, activar/desactivar banderas de movimiento, encender/apagar luces, iniciar o reiniciar animaciones, y alternar recorridos automáticos de cámara o animaciones de objetos. Por ejemplo, la barra espaciadora alterna el estado de una luz, la tecla 'R' activa un recorrido automático, y otras teclas inician animaciones específicas (como el rebote del laboratorio o animaciones de sillas). Además, registra el estado de las teclas en un arreglo para controles continuos. La función `MouseCallback` gestiona el movimiento del ratón para actualizar la dirección de la cámara, permitiendo rotar la vista de la escena 3D en función del movimiento del mouse.

```

void KeyCallback(GLFWwindow* window, int key, int scancode, int action, int mode)
{
    if (GLFW_KEY_ESCAPE == key && GLFW_PRESS == action)
    {
        glfwSetWindowShouldClose(window, GL_TRUE);
    }
    if (key >= 0 && key < 1024)
    {
        if (action == GLFW_PRESS)
        {
            keys[key] = true;
        }
        else if (action == GLFW_RELEASE)
        {
            keys[key] = false;
        }
    }

    if (keys[GLFW_KEY_SPACE])
    {
        active = !active;
        if (active)
        {
            Light1 = glm::vec3(1.0f, 1.0f, 0.0f);
        }
        else
        {
            Light1 = glm::vec3(0); //Cuado es solo un valor en los 3 vectores
            pueden dejar solo una componente
        }
    }

    if (key == GLFW_KEY_R && action == GLFW_PRESS) {
        recorridoActivo = !recorridoActivo;
    }
}

```

```

if (key == GLFW_KEY_T && action == GLFW_PRESS) { recorridoActivo2 =
    !recorridoActivo2;
    recorridoActivo = false; // Desactiva el otro recorrido si está activo
}

if (key == GLFW_KEY_M && action == GLFW_PRESS) {
    animacionActivaLabNuevo = true; animacionTerminada = false;
    enRebote = true; impactoFinal =
    false;
    faseAplastamientoActiva = false; numRebotes
    = 0;
    escalaY_LabViejo = 1.0f;
    alphaLabViejo = 1.0f;
    posLabNuevo = glm::vec3(0.0f, 100.0f, 0.0f); velocidadY = 0.0f;
}

if (key == GLFW_KEY_L && action == GLFW_PRESS) {
    // Activa la primera animación
    animarSillasViejas1 = true; frameSillasViejas1
    = 0;
    delayRemolino1 = 0;

    // Prepara el desfase de las otras dos animarSillasViejas2 =
    false; frameSillasViejas2 = 0;
    delayRemolino2 = 0; animarSillasViejas3 =
    false; frameSillasViejas3 = 0;
    delayRemolino3 = 0;
}

if (key == GLFW_KEY_J && action == GLFW_PRESS) { labYaAterrizo =
    false; prepararKeyframesCaidaSillasNuevas();

    // Activa la primera animación de sillas nuevas animarSillasNuevas1 =
    true;
    frameSillasNuevas1 = 0;
    delayCaida1 = 0;

    // Prepara el desfase de las otras dos animarSillasNuevas2 =
    false; frameSillasNuevas2 = 0;
    delayCaida2 = 0; animarSillasNuevas3 = false;
    frameSillasNuevas3 = 0;
    delayCaida3 = 0;
}
}

void MouseButton(GLFWwindow* window, double xPos, double yPos)
{
    if (firstMouse)
    {
        lastX = xPos;
        lastY = yPos;
        firstMouse = false;
    }

    GLfloat xOffset = xPos - lastX;
    GLfloat yOffset = lastY - yPos; // Reversed since y-coordinates go from bottom to left

    lastX = xPos;
    lastY = yPos;
    camera.ProcessMouseMove(xOffset, yOffset);
}

```

Experiencia adquirida

- **Luis 317290967**

During the development of the new version of the microcomputer lab, I learned a lot about the animation and movement of different 3D models and objects as I progressed through my computer graphics classes. At first, I had a lot of difficulty managing the paths a model can follow, especially when it needed to take different directions or return. However, with time, practice, and taking detailed notes to monitor every turn and distance, you'll gain an understanding of how to work better with the tools and, in particular, with the different animation variables that can be used.

You'll also learn how to animate bodies more naturally, adjusting their movements to achieve more visually appealing effects for certain models. Another important aspect was the implementation of instructions: learning how to make a model perform different actions depending on the key pressed, from a single movement to a combination of several.

Beyond technical skills, this project also helped me improve my time management, organize my work, and define the duration of each function, always choosing the best option for each instruction. Finally, I strengthened my communication with the team, as their collaboration was key in generating ideas, making decisions, and making the necessary adjustments to meet our goals

- **Diego 319296738**

During this project, I learned a lot about the Blender tool, exploring its potential for modeling, texturing, and animating 3D objects. At first, it seemed complex, but over time I understood how to work with the different tools it offers, from creating models to implementing materials and lights.

I also learned how to light a scene, both in OpenGL and in Blender. I discovered the importance of lighting to bring realism to an environment, adjusting intensities, colors, and types of lights to achieve visually appealing effects.

One of the most important lessons was learning how to animate objects using keyframes, applying this in both Blender and OpenGL. This allowed me to bring the models to life, creating smooth and coordinated movements, and understanding how to manipulate transformations over time.

Besides the technical skills, this project helped me grow personally. I learned how to be a leader, to organize work, and make decisions. I also greatly improved

my ability to communicate with my team, share ideas, solve doubts, and collaborate to achieve our goals.

- **Eduardo 319281998**

Completing this project was a great learning experience for me, as it allowed me to deepen and apply my knowledge in areas ranging from graphics programming to 3D modeling and animation. Throughout the development process, I faced various technical challenges, especially when working with OpenGL and implementing animations using keyframes, which helped me improve my skills in structuring and optimizing code in complex graphical environments.

I found it very interesting to see how every small adjustment in the animations, textures, or lighting could transform the visual experience and make it more realistic and dynamic. By working on the integration of interactive elements, I was able to better understand the importance of attention to detail and the logic behind user interaction with the virtual environment. Additionally, I had to learn how to solve problems on the fly, search for documentation, and try out different solutions, which strengthened my ability to face technical challenges and adapt to new tools.

Another important aspect was personal organization and time management, as I had to balance several tasks at once, such as project documentation, programming, and 3D modeling. In short, this project allowed me to grow both technically and personally, and it was very satisfying to see how all the parts came together in a visually attractive and functional result.

Referencias

- Microsoft. (s.f.). *Visual Studio Code documentation*. <https://code.visualstudio.com/docs>
- Blender Documentation Team. (2025). *Blender 4.4 Reference Manual*. <https://docs.blender.org/manual/en/latest/>
- Atlassian. (2025). *Trello*. <https://trello.com/es>
- Ramírez, D. (2025). *PF_Laboratorio* [Repositorio GitHub]. https://github.com/DiegoRamirez25/PF_Laboratorio/tree/main

- Creative Commons. (2013). *Attribution 4.0 International (CC BY 4.0) License*. <https://creativecommons.org/licenses/by/4.0/>
- Sketchfab. (2025). *Sketchfab*. <https://sketchfab.com/>
- MDN Web Docs. (2024). *@keyframes*. <https://developer.mozilla.org/es/docs/Web/CSS/@keyframes>